

PORT Driver User Guide

Traveo™ II Family

About this document

Scope and purpose

This user guide describes the architecture, configuration, and use of the PORT Driver. This guide also explains the functionality of the driver and provides a reference to the driver's API.

The installation, build process, and general information about the use of the EB tresos Studio are not within the scope of this document. Refer to the *EB tresos Studio for ACG8 User's Guide* [6] for a detailed description of these topics.

Intended audience

This document is intended for anyone who uses the PORT driver of the Traveo II family.

Document Structure

Chapter **1 General Overview** gives a brief introduction to the PORT Driver, explains the embedding of the driver in the AUTOSAR environment, and describes the supported hardware and development environment.

Chapter **2 Using the PORT Driver** details the steps required to use the PORT Driver in your application.

Chapter **3 Structure and Dependencies** describes the file structure and the dependencies for the PORT Driver.

Chapter **4 EB tresos Studio Configuration Interface** describes the driver's configuration with the EB tresos Studio.

Chapter **5 Functional Description** gives a functional description of all services offered by the PORT Driver.

Chapter **6 Hardware Resources** describes the hardware resources used.

The Appendix provides a complete API reference and Access Register Table.

Abbreviations and definitions

Table 1 **Abbreviations**

Words and Terms	Description
API	Application Programming Interface
ASIL	Automotive Safety Integrity Level
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software. Standardized part of software which does not fulfill a vehicle functional job.
DEM	Diagnostic Event Manager
DET	Default Error Tracer
EB tresos Studio	Elektrobit Automotive configuration framework
HSIOM	High-Speed I/O Matrix
HW	Hardware

About this document

Words and Terms	Description
MCAL	Microcontroller Abstraction Layer
MCU	Microcontroller Unit
OS	Operating System
SMART I/O	Programmable I/O
SW	Software
UTF-8	8-Bit Universal Character Set Transformation Format
μC	Microcontroller

Related Documents

AUTOSAR Requirements and Specifications

Bibliography

- [1] General Specification of Basic Software Modules, AUTOSAR Release 4.2.2.
- [2] Specification of PORT Driver, AUTOSAR Release 4.2.2.
- [3] Specification of Standard Types, AUTOSAR Release 4.2.2.
- [4] Specification of ECU Configuration Parameters, AUTOSAR Release 4.2.2.
- [5] Specification of Default Error Tracer, AUTOSAR Release 4.2.2.

Elektrobit Automotive Documentation

Bibliography

- [6] EB tresos Studio for ACG8 User's Guide.

Hardware Documentation

The hardware documents are listed in the delivery notes.

Related Standards and Norms

Bibliography

- [7] Layered Software Architecture, AUTOSAR Release 4.2.2.

Table of contents

About this document	1
Table of contents	3
1 General Overview	6
1.1 Introduction to the PORT Driver	6
1.2 User Profile	6
1.3 Embedding in the AUTOSAR Environment	7
1.4 Supported Hardware	8
1.5 Development Environment	8
1.6 Character Set and Encoding	8
2 Using the PORT Driver	9
2.1 Installation and Prerequisites	9
2.2 Configuring the PORT Driver	9
2.2.1 Architecture Specifics	10
2.3 Adapting your Application	10
2.4 Starting the Build Process	12
2.5 Measuring Stack Consumption	12
2.6 Memory Mapping	12
2.6.1 Memory Allocation Keyword	12
3 Structure and Dependencies	14
3.1 Static Files	14
3.2 Configuration Files	14
3.3 Generated Files	14
3.4 Dependencies	15
3.4.1 DET	15
3.4.2 Resource File	15
3.4.3 BSW Scheduler	15
3.4.4 Error Callout Handler	15
4 EB tresos Studio Configuration Interface	16
4.1 General Configuration	16
4.2 Port Configuration Set	16
4.3 Port Configuration	16
4.4 Port Pin Configuration	17
4.5 AMUX Splitter Cell Configuration	19
4.6 Port Default Configuration Set	19
4.7 Port Default Configuration	19
4.8 Port Default Pin Configuration	20
4.9 Port Default AMUX Splitter Cell Configuration	21
4.10 Port Trigger Configuration Set	21
4.11 Port Trigger Group Configuration	22
4.12 Port Output Trigger Configuration	22
4.13 Port Trigger 1-to-1 Group Configuration	22
4.14 Port 1-to-1 Output Trigger Configuration	23
5 Functional Description	24
5.1 Inclusion	24
5.2 Initialization	24
5.3 Runtime Reconfiguration	24
5.4 Ports and Port Pins	26

Table of contents

5.5	Trigger Command	26
5.6	API Parameter Checking	26
5.7	Configuration Checking	28
5.8	Re-entrancy	30
5.9	Debugging Support	30
6	Hardware Resources.....	31
6.1	Ports and Pins.....	31
6.2	SMART I/O Blocks	31
6.3	AMUX Splitter Cells.....	31
6.4	Trigger Groups and Trigger 1-to-1 Groups	31
6.5	Timer.....	31
6.6	Interrupts.....	31
7	Appendix	32
7.1	API Reference.....	32
7.1.1	Data Types	32
7.1.1.1	Port_ConfigType	32
7.1.1.2	Port_PinType.....	32
7.1.1.3	Port_PinModeType	32
7.1.1.4	Port_PinDirectionType	32
7.1.1.5	Port_PinLevelValueType.....	33
7.1.1.6	Port_StatusType	33
7.1.1.7	Port_AmuxCellType	35
7.1.1.8	Port_AmuxSplitCtlStatusType.....	35
7.1.1.9	Port_TriggerGroupIdType	36
7.1.1.10	Port_TriggerIdType.....	36
7.1.1.11	Port_TriggerSensitiveType	36
7.1.1.12	Port_TriggerActivationType	37
7.1.1.13	Port_TriggerIdStatusType	37
7.1.1.14	Port_TriggerCmdStatusType.....	37
7.1.2	Constants.....	38
7.1.2.1	Error Codes.....	38
7.1.2.2	Version Information	39
7.1.2.3	Module Information	39
7.1.2.4	API Service IDs	39
7.1.2.5	Port Pin Status	40
7.1.2.6	AMUX Splitter Cell Status.....	45
7.1.2.7	Trigger Status	45
7.1.3	Functions	46
7.1.3.1	Port_Init.....	46
7.1.3.2	Port_SetPinDirection	46
7.1.3.3	Port_RefreshPortDirection	47
7.1.3.4	Port_GetVersionInfo	48
7.1.3.5	Port_SetPinMode	48
7.1.3.6	Port_GetStatus.....	49
7.1.3.7	Port_GetAmuxSplitCtlStatus.....	50
7.1.3.8	Port_SetToDioMode.....	50
7.1.3.9	Port_SetToAlternateMode.....	51
7.1.3.10	Port_SetTrigger.....	52
7.1.3.11	Port_ActTrigger	53
7.1.3.12	Port_DeactTrigger.....	54

Table of contents

7.1.3.13	Port_GetTriggerIdStatus.....	54
7.1.3.14	Port_GetTriggerCmdStatus.....	55
7.1.4	Required Callback Functions.....	56
7.1.4.1	DET.....	56
7.1.4.2	Callout functions.....	56
8	Appendix	58
8.1	Access Register Table	58
8.1.1	HSIOM	58
8.1.2	GPIO	59
8.1.3	SMARTIO	61
8.1.4	PERI.....	62
	Revision history	63

1 General Overview

1.1 Introduction to the PORT Driver

The PORT Driver is a set of software routines for initializing the whole port structure of the microcontroller.

Many port pins can be assigned to various functionalities such as the following:

- GPIO
- ADC
- CAN
- ICU
- PWM
- SPI

For this purpose, the PORT Driver provides configuration options for each port pin for the following:

- Mode
- Direction
- Level value
- Direction changeable flag
- Mode changeable flag

The driver conforms to the AUTOSAR standard and is implemented according to the *Specification of PORT Driver* [\[2\]](#).

Furthermore, the PORT Driver is delivered with a plugin for the EB tresos Studio, which allows the user to statically configure the driver. It provides an interface to define symbolic names and the functionality of all port pins used in GPIO or alternative mode.

In addition, the PORT Driver provides software routines for initializing the High-Speed I/O Matrix (HSIOM), the Programmable I/O (SMARTIO), and the Trigger multiplexers. The Trigger multiplexers control the communication between peripherals.

For this purpose, the PORT Driver provides configuration options for the following:

- HSIOM setting
- SMARTIO setting
- Trigger group
- Trigger 1-to-1 group

1.2 User Profile

This guide is intended for users with a basic knowledge of the following:

- Embedded systems
- The C programming language
- The AUTOSAR standard
- The target hardware architecture

1.3 Embedding in the AUTOSAR Environment

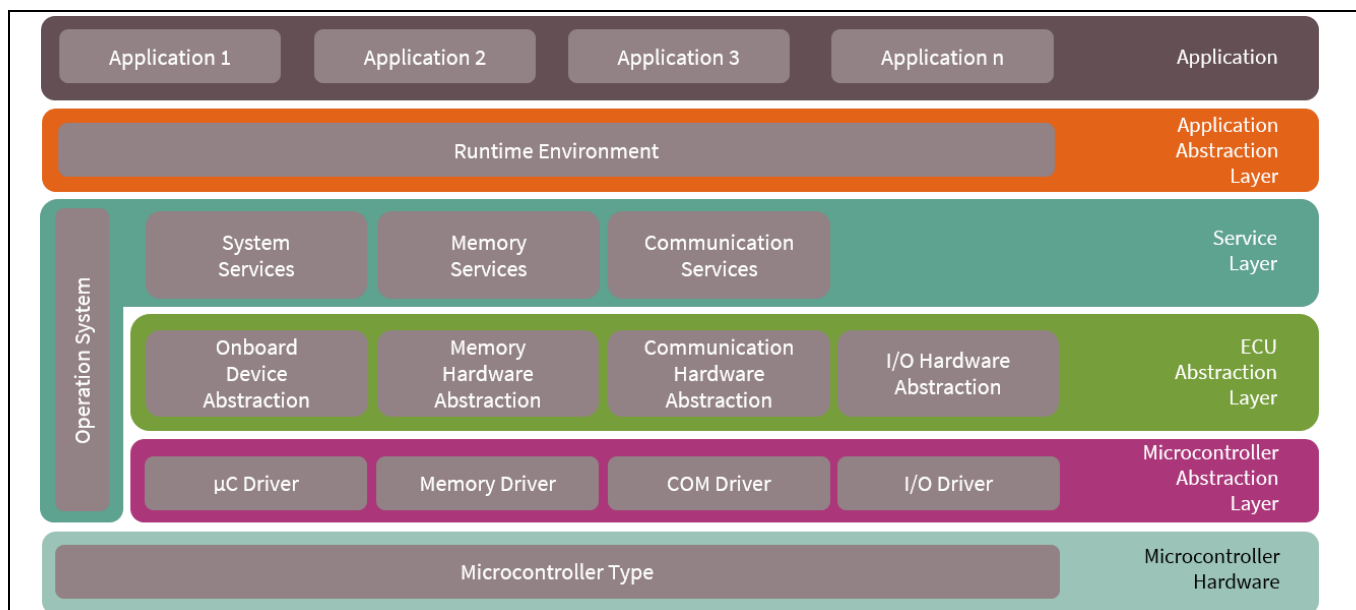


Figure 1 Overview of AUTOSAR Software Layers

Figure 1 depicts the layered AUTOSAR software architecture. The PORT Driver (**Figure 2**) is part of the Microcontroller Abstraction Layer (MCAL), the lowest layer of Basic Software in the AUTOSAR environment.

As an internal I/O driver, it provides a standardized and microcontroller-independent interface to higher software layers for accessing ports and port pins of the ECU hardware.

For a thorough overview of the AUTOSAR-layered software architecture, refer to *Layered Software Architecture* [7].

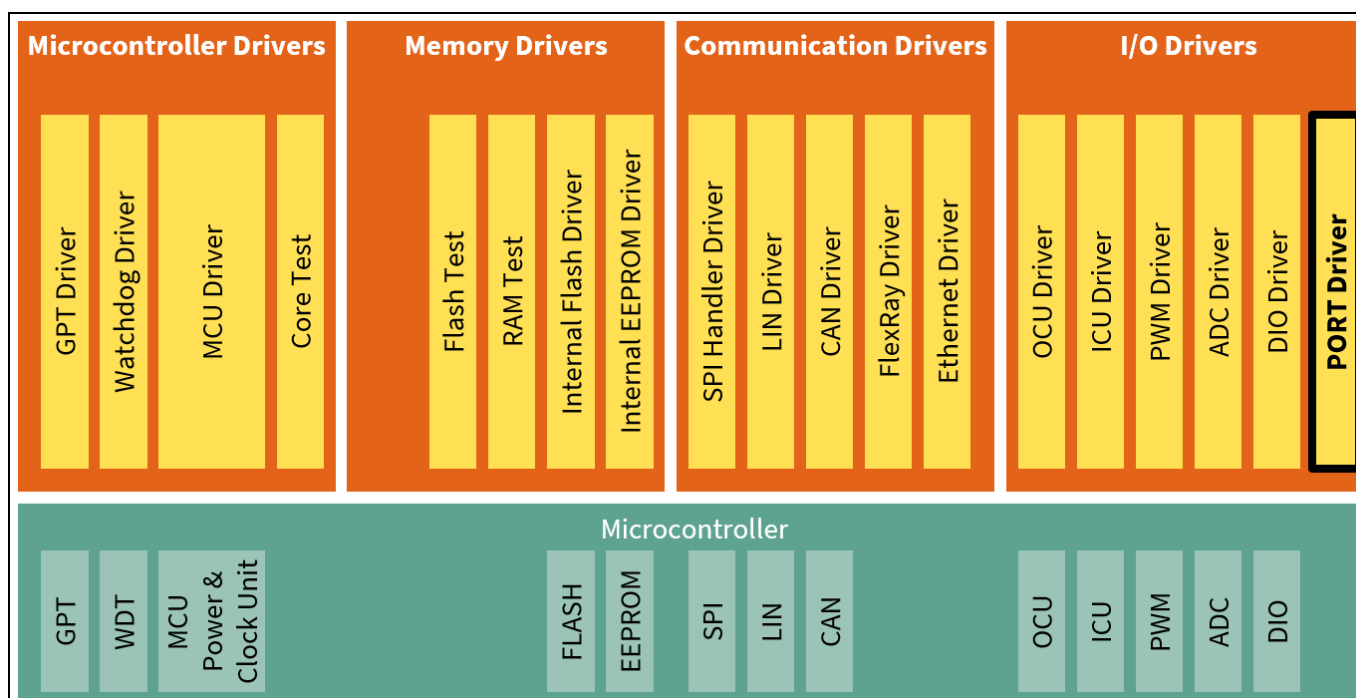


Figure 2 PORT Driver in MCAL Layer

1.4 Supported Hardware

This version of the PORT Driver supports the Traveo II Microcontroller. The supported derivatives are listed in the release notes.

Smaller derivatives have only a subset of the available port pins. Additional derivatives can be implemented or supported via a resource file without change of the PORT Driver and/or this document. Refer to the Resource plugin for a list of all supported derivatives.

1.5 Development Environment

The development environment corresponds to AUTOSAR release 4.2.2. The modules BASE, Make, and Resource are needed for proper functionality of the PORT Driver.

1.6 Character Set and Encoding

All source code files of the PORT Driver are restricted to the ASCII character set. The files are encoded in UTF-8 format, with only the 7-bit subset (values 0x00 ... 0x7F) being used.

2 Using the PORT Driver

This chapter describes all necessary steps to incorporate the PORT Driver into your application.

2.1 Installation and Prerequisites

Note: Before you start, see the *EB tresos Studio for ACG8 User's Guide* [6] for the following information.

1. The installation procedure of EB tresos ECU AUTOSAR components.
2. The usage of the EB tresos Studio.
3. The usage of the EB tresos ECU AUTOSAR Build Environment (It includes an explanation of how to setup and integrate your application within the EB tresos ECU AUTOSAR Build Environment).

The installation of the PORT Driver complies with the general installation procedure for EB tresos ECU AUTOSAR components given in the documents mentioned above. If the driver is successfully installed, it will appear in the module list of the EB tresos Studio (see *EB tresos Studio for ACG8 User's Guide* [6]).

In the following section, it is assumed that the project is properly set up and is using the application template as described in the *EB tresos Studio for ACG8 User's Guide* [6]. This template provides the necessary folder structure, project, and Makefiles needed to configure and compile your application within the build environment. You need to be familiar with the usage of the command shell.

2.2 Configuring the PORT Driver

This section gives a brief overview of the configuration structure defined by AUTOSAR to use the PORT Driver.

The following container is used to configure common behavior.

- *PortGeneral*: This container is mainly used to restrict/extend the API of the PORT module and enable/disable default error trace.

For detailed information and description, see Chapter 4 **EB tresos Studio Configuration Interface**.

Note: Be sure that your application also includes an AUTOSAR compliant DET when default error detection is enabled. Otherwise your application will not compile if the option **Default Error Detection** is switched on.

A value for the following characteristics needs to be given for each configured or required port pin:

- PortPinDirection
- PortPinLevelValue
- PortPinDirectionChangeable at runtime
- PortPinModeChangeable at runtime

The *PortPin* container describes the information of the individual port pin. It has a user-changeable symbolic name which can be used in the application.

The PORT Driver initializes the whole port structure of the MCU. All port pins that are not configured are initialized with the value in Port Default Pin Configuration. For detailed information and description, see **4.8 Port Default Pin Configuration**.

2.2.1 Architecture Specifics

Refer to Chapter [4 EB tresos Studio Configuration Interface](#), where all configuration parameters are described.

2.3 Adapting your Application

To use the PORT Driver in your application, you must first include the PORT Driver header file by adding the following code line to your source file:

```
#include "Port.h" /* PORT Driver */
```

This publishes all needed function/data prototypes and symbolic names of the configuration into the application.

You must also implement the error callout function for ASIL safety extension.

Declare the error callout function in the specified file by the `PortIncludeFile` parameter and implement in your application (see [7.1.4 Required Callback Functions](#), Error callout API).

The error callout function name can be configured by the `PortErrorCalloutFunction` parameter.

The next step is to initialize and configure the ports and port pins. Chapter [4 EB tresos Studio Configuration Interface](#) explains how to configure the port with the PORT Driver in the EB tresos Studio. The PORT module will be automatically enabled if an appropriate parameter configuration of the PORT module is available in your application.

The port initialization can be done (in an initialization task) with the following function call and parameter.

```
Port_Init(&Port_Config[0]);
```

API functions can be called with the symbolic names (for example, `PortConf_PortPin_MY_IO_PIN`, `PortConf_PortTrGroupContainer_MY_TRIGGER_GROUP`, `PortConf_PortOutputTrigger_MY_OUTPUT_TRIGGER`, `PortConf_PortTrlToIGroupContainer_MY_TRIGGER_1TOI_GROUP` and `PortConf_PortlToIOutputTrigger_MY_1TOI_OUTPUT_TRIGGER`) from your configuration after `Port_Init()`.

For more information about the `Port_SetPinDirection` function and other services, see Chapter [5 Functional Description](#).

```
Port_SetPinDirection(PortConf_PortPin_MY_IO_PIN, PORT_PIN_OUT);
```

The pin mode can be changed at runtime using the `Port_SetPinMode()` function. The user is responsible to pass the architecture-specific mode value listed the datasheet for the selected subderivative. A list of all available modes can be found in the `/generated/include/Port_Cfg_Der.h` file.

```
Port_SetPinMode(PortConf_PortPin_MY_IO_PIN, PORT_PIN_MODE_P000_0_GPIO);
```

The pin mode can be switched between DIO mode (`PORT_PIN_MODE_GPIO`) and configured mode at runtime using the `Port_SetToDioMode()` and `Port_SetToAlternateMode()` functions. If the pin's configured mode is DIO mode, these APIs have no effect.

```
/* Switch from the configured mode to DIO mode */
Port_SetToDioMode(PortConf_PortPin_MY_IO_PIN);
/* Switch from DIO mode to the configured mode */
Port_SetToAlternateMode(PortConf_PortPin_MY_IO_PIN);
```

The trigger setting can be changed at runtime using the `Port_SetTrigger()` function. The user is responsible to pass the architecture-specific input trigger listed the datasheet for the selected subderivative. A list of all available input triggers can be found in the `/generated/include/Port_Cfg_Der.h` file.

```
Port_SetTrigger(PortConf_PortTrGroupContainer_MY_TRIGGER_GROUP,
               PortConf_PortOutputTrigger_MY_OUTPUT_TRIGGER,
               PORT_TR_GROUP_DW0_TR_IN_CPUS_DW0_TR_OUT_0,
               FALSE, PORT_TR_SENSITIVE_LEVEL, FALSE);
```

Users can control trigger activation with Trigger Command at runtime. The specified output trigger can be activated or deactivated using the `Port_ActTrigger()` and `Port_DeactTrigger()` functions. The Trigger Command functionality can control only one trigger activation. This feature cannot be used by multiple tasks or modules at the same time. The status of the Trigger Command can be acquired using the `Port_GetTriggerCmdStatus()` function.

```
/* If trcmd_status.activate = 1U, Trigger Command is not available because used by
other tasks or modules. */
Port_GetTriggerCmdStatus(&trcmd_status);

/* If the sensitive type is set to PORT_TR_SENSITIVE_LEVEL, the specified output
trigger / 1-to-1 output trigger continues the activated status until calling
Port_DeactTrigger(). */
Port_ActTrigger(PortConf_PortTrGroupContainer_MY_TRIGGER_GROUP,
               PortConf_PortOutputTrigger_MY_OUTPUT_TRIGGER,
               PORT_TR_ACTIVATION_OUTPUT,
               PORT_TR_SENSITIVE_LEVEL);

(PortConf_PortOutputTrigger_MY_OUTPUT_TRIGGER is being activated)
/* The activating output trigger is deactivated */
Port_DeactTrigger();

/* If the sensitive type is set to PORT_TR_SENSITIVE_EDGE, the specified output
trigger is activated for two "clk_peri" cycles. Port_DeactTrigger does not have to
be called. After generating two cycle pulse, the output trigger is deactivated by
hardware. */
Port_ActTrigger(PortConf_PortTrGroupContainer_MY_TRIGGER_GROUP,
               PortConf_PortOutputTrigger_MY_OUTPUT_TRIGGER,
               PORT_TR_ACTIVATION_OUTPUT,
               PORT_TR_SENSITIVE_EDGE);

/* If setting the input trigger for Trigger Command, the multiple output triggers
connected to the specified input trigger can be activated at the same time. */
Port_ActTrigger(PortConf_PortTrGroupContainer_MY_TRIGGER_GROUP,
               PORT_TR_GROUP_DW0_TR_IN_CPUS_DW0_TR_OUT_0,
               PORT_TR_ACTIVATION_INPUT,
               PORT_TR_SENSITIVE_EDGE);
```

Additional functions:

- `Port_RefreshPortDirection();`
- `Port_GetVersionInfo(Std_VersionInfoType*);`
- `Port_GetStatus(PortConf_PortPin_MY_IO_PIN, Port_StatusType*);`
- `Port_GetAmuxSplitCtlStatus(PortConf_PortAmuxSplitCell_MY_CELL, Port_AmuxSplitCtlStatusType*);`
- `Port_GetTriggerIdStatus(PortConf_PortTrGroupContainer_MY_TRIGGER_GROUP, PortConf_PortOutputTrigger_MY_OUTPUT_TRIGGER, Port_TriggerIdStatusType*);`

These functions can be used wherever needed.

2.4 Starting the Build Process

Do the following to build your application:

Note: For a clean build, use the build command with target `clean_all` before (`make clean_all`).

1. On the command shell, type the following command to generate the necessary configuration-dependent files. See [3.3 Generated Files](#):

```
> make generate
```

2. Type the following command to resolve the required file dependencies:

```
> make depend
```

3. Type the following command to compile and link the application:

```
> make (optional target: all)
```

The application is now built. All files are compiled and linked to a binary file, which can be downloaded to the target hardware.

2.5 Measuring Stack Consumption

Do the following to measure stack consumption. It requires the Base module for proper measurement.

Note: All files (including library files) should be rebuilt with dedicated compiler option.

Note: The executable file built in this step must be used only to measure stack consumption.

1. Add the following compiler option to the Makefile to enable stack consumption measurement.

```
-DSTACK_ANALYSIS_ENABLE
```

2. Type the following command to clean library files.

```
> make clean_lib
```

3. Follow the build process described in [2.4 Starting the Build Process](#).
4. Follow the instructions in the release notes and measure the stack consumption.

2.6 Memory Mapping

The `Port_MemMap.h` file in the `$(TRESOS_BASE)/plugins/MemMap_TS_T40D13M0I0R0/include` directory is a sample. This sample file is replaced by the file generated by the MEMMAP module. Input to the MEMMAP module is generated as `Port_Bswmd.xml` in the `$(PROJECT_ROOT)/output/generated/swcd` directory of your project folder.

2.6.1 Memory Allocation Keyword

- `PORT_START_SEC_CODE_ASIL_B` / `PORT_STOP_SEC_CODE_ASIL_B`

The memory section type is CODE. All executable code is allocated in this section.

- `PORT_START_SEC_CONST_ASIL_B_UNSPECIFIED` / `PORT_STOP_SEC_CONST_ASIL_B_UNSPECIFIED`

The memory section type is CONST. The following constants are allocated in this section:

- Port Configuration data

Using the PORT Driver

- Hardware register base address data
- PORT_START_SEC_VAR_INIT_ASIL_B_UNSPECIFIED / PORT_STOP_SEC_VAR_INIT_ASIL_B_UNSPECIFIED

The memory section type is VAR. The following variable is allocated in this section:

- Pointer to the configuration data

3 Structure and Dependencies

The PORT Driver consists of a static configuration and generated files.

3.1 Static Files

`$(PLUGIN_PATH)=$(TRESOS_BASE)/plugins/Port_TS_*` is the path to the PORT module plugin.

`$(PLUGIN_PATH)/lib_src` contains all static source files of the PORT Driver. These files contain the functionality of the driver, which does not depend on the current configuration. The files are grouped into a static library.

`$(PLUGIN_PATH)/src` contains configuration dependent source files or special derivate files. Each file will be built again when the configuration is changed.

All necessary source files will be automatically compiled and linked during the build process and all include paths will be set if the PORT Driver is enabled.

`$(PLUGIN_PATH)/include` is the basic public include directory that you need to include *Port.h*.

`$(PLUGIN_PATH)/autosar` directory contains the AUTOSAR ECU parameter definition with vendor, architecture and derivate specific adaptations to create a correct matching parameter configuration for the PORT module.

3.2 Configuration Files

The configuration of the PORT Driver is done with the EB tresos Studio. When saving a project, the configuration description is written to the file *Port.xdm*. It is located under `$(PROJECT_ROOT)/config` in your project folder. This file serves as input for the generation of the configuration-dependent source and header files during the build process.

3.3 Generated Files

During the build process, the following files are generated based on the current configuration description. These files are in the sub folder `output/generated` of your project folder.

- *include/Port_Cfg.h* and *include/Port_Cfg_Der.h* define all by the API needed symbolic names for configured port pins and will be included by *Port.h*. Settings are separated into architecture and derivate specific.
- *include/Port_Cfg_Der_Internal_User.h* defines the configuration for the user configured port pins.
- *include/Port_Cfg_Der_Internal_Defaults.h* defines the subderivative specific default values for all pins. Not in *Port_Cfg_Der_Internal_User.h* configured pins are added and used to create a complete port configuration structure to initialize the whole port structure of the MCU.
- *include/Port_Cfg_Include.h* defines the custom include headers generated from the configuration.
- *src/Port_PBcfg.c* contains the constant structure for the PORT configuration.
- *src/Port_PBcfg_Der.c* contains the error check function dependent on hardware.

Note: You do not need to add the generated source files to your application make file. They will be compiled and linked automatically during the build process.

- *swcd/Port_Bswmd.arxml* contains BswModuleDescription.

Note: Additional steps are required for the generation of BSW module description.
In EB tresos Studio, follow the menu path **Project > Build Project** and select **generate_swcd**.

3.4 Dependencies

3.4.1 DET

If the default error detection is enabled in the PORT module configuration, DET needs to be installed, configured, and built into the application.

3.4.2 Resource File

The PORT Driver needs registered and configured Resource module. The PORT Driver produces lots of errors such as “port pin not available in this derivative” or “*PortPinMode/PortPinDirection* not possible” if this dependency is not resolved properly.

3.4.3 BSW Scheduler

The Port Handler/Driver uses the following services of the BSW Scheduler to enter and leaves critical sections:

- SchM_Enter_Port_PORT_EXCLUSIVE_AREA_0(void)
- SchM_Exit_Port_PORT_EXCLUSIVE_AREA_0(void)

Make sure that the BSW Scheduler is properly configured and initialized before using the Port Handler/Driver.

3.4.4 Error Callout Handler

The error callout handler is called on every error that is detected regardless of whether the default error detection is enabled or disabled. The error callout handler is an ASIL safety extension that is not specified by AUTOSAR. It is configured via the configuration parameter `PortErrorCalloutFunction`.

4 EB tresos Studio Configuration Interface

The GUI is not part of the current delivery. For further information see *EB tresos Studio for ACG8 User's Guide* [6].

4.1 General Configuration

- `PortDevErrorDetect` enables or disables the default error notification for the PORT module.

Setting this parameter to false will disable the notification of development errors via DET. However, in contrast to AUTOSAR specification, detection of development errors is still enabled as safety mechanisms (fault detection).

- `PortSetPinDirectionApi` enables or disables the function `Port_SetPinDirection`.
- `PortSetPinModeApi` enables or disables the function `Port_SetPinMode`.
- `PortVersionInfoApi` enables or disables the function `Port_GetVersionInfo`.
- `PortSafetyFunctionApi` enables or disables the functions `Port_GetStatus`, `Port_GetAmuxSplitCtlStatus` and `Port_GetTriggerIdStatus`.
- `PortSetToDioAlternateModeApi` enables or disables the functions `Port_SetToDioMode` and `Port_SetToAlternateMode`.
- `PortSetTriggerApi` enables or disables the functions `Port_SetTrigger`.
- `PortTriggerCommandApi` enables or disables the functions `Port_ActTrigger`, `Port_DeactTrigger` and `Port_GetTriggerCmdStatus`.
- `PortErrorCalloutFunction` is used to specify the error callout function name. The function is called on every error. The ASIL level of this function limits the ASIL level of the PORT Driver.

Note: `PortErrorCalloutFunction` must be valid C function name, otherwise an error would occur in configuration phase.

- `PortIncludeFile` lists the file names that shall be included within the driver. Any application specific symbol that is used by the Port configuration (e.g. error callout function) should be included by configuring this parameter.

Note: `PortIncludeFile` must be a filename with extension `.h` and a unique name, otherwise some errors would occur in configuration phase.

4.2 Port Configuration Set

Port Configuration is used as configuration pointer parameter of `Port_Init()`. The symbolic name is derived from the `PortConfigSet` container short name prefixed with `"PortConf_PortConfigSet_"`.

The value is configuration data (that is, `Port_Config[0]`, `Port_Config[1]`) and defined as the user changeable symbolic name which can be used in the application.

- `PortTriggerReference` refers to the `PortTriggerConfigSet` for this `PortConfigSet`.

4.3 Port Configuration

For the configuration of the Ports the following parameters are available:

- `PortNumberOfPortPins` is the number of specified *PortPins* in this *PortContainer*.
- `PortId` is the Port Id in this *PortContainer*.

- `PortSmartioClockSource` is the source of clock ("clk_fabric") and reset ("rst_fabric_n") for the SMART I/O block.
- If `PortSmartioHoldOverrideEnable` is checked, the SMART I/O controls the IO cell hold override functionality for the SMART I/O block. If not checked, it is controlled by the HSIOM.
- If `PortSmartioPipelineEnable` is checked, the pipeline setting is enabled for the SMART I/O block.
- If `PortSmartioEnable` is checked, the programmable I/O setting is enabled for the SMART I/O block.
- `PortSmartioDataUnitTr0Source` is the data unit input signal "tr0_in" source for the SMART I/O block. See [Table 18](#) for selectable values.
- `PortSmartioDataUnitTr1Source` is the data unit input signal "tr1_in" source for the SMART I/O block. See [Table 18](#) for selectable values.
- `PortSmartioDataUnitTr2Source` is the data unit input signal "tr2_in" source for the SMART I/O block. See [Table 18](#) for selectable values.
- `PortSmartioDataUnitData0Source` is the data unit input data "data0_in" source for the SMART I/O block. See [Table 19](#) for selectable values.
- `PortSmartioDataUnitData1Source` is the data unit input data "data1_in" source for the SMART I/O block. See [Table 19](#) for selectable values.
- `PortSmartioDataUnitBitSize` is the size/width of the data unit data operands (in bits) for the SMART I/O block.
- `PortSmartioDataUnitOperation` is the data unit opcode specifies the data unit operation for the SMART I/O block. See [Table 20](#) for selectable values.
- `PortSmartioDataUnitSource` is the data unit input data source for the SMART I/O block.

Note: *It is able to configure some PortContainers into a PortConfigSet. And only one port can be configured in one PortContainer. Therefore, configure the required PortContainers for each port and create only those PortPins which you want to use in each PortContainer.*

Note: *The parameters for the SMART I/O block are editable only if the SMART I/O functionality of the specified port is valid.
For more detail about SMART I/O configuration, see the hardware manual.*

4.4 Port Pin Configuration

For configuration of the Port Pins, the following parameters are available:

- `PortPinDirection` is the initial direction of the pin (IN, OUT or IN/OUT disabled).
`PORT_PIN_IN_OUT_DISABLED` is for Analog I/O. It can be set if `PortPinInitialMode` is AMUXA, AMUXB, AMUXA_DSI, or AMUXB_DSI. See [Port_PinDirectionType](#) for selectable values.

Note: *If the pin is used in the AMUXA, AMUXA_DSI, AMUXB, or AMUXB_DSI mode, the pin direction must be set to `PORT_PIN_IN_OUT_DISABLED`. In case of other modes, the pin direction must be set to `PORT_PIN_IN` or `PORT_PIN_OUT`.*

- If `PortPinDirectionChangeable` is checked, the direction is changeable on a port pin during runtime.
- `PortPinId` is the pin ID of the port pin.
 - This value will be assigned to the symbolic name derived from the port pin container short name.
- `PortPinName` is port pin name.
 - The symbolic name derived from the *PortPin* container, a short name prefixed with "PortConf_PortPin_".

For example, the appropriate `PortPinName` for Port 0 Pin 1 or Port 22 Pin 7 are `P000_1` and `P022_7`.

- `PortPinInitialMode` is the initial mode of the port pin. The allowed modes for a certain pin are hardware dependent. Note that `PortPinInitialMode` influences the input/output behavior.
- If `PortPinModeChangeable` is checked, the mode is changeable on a port pin during runtime.
- `PortPinLevelValue` is the level value from the Port pin and can be configured with the following values with each `PortPin`. See [Port_PinLevelValueType](#) for selectable values.

Note: *When the pin mode is set to other than GPIO, the pin output shall be controlled by the peripheral functionality.
Therefore, if the initialized pin mode is not GPIO, the configuration parameter `PortPinLevelValue` is ignored.*

- `PortPinOutputDrive` is the output drive mode of the port pin and can be configured the available output drive with each `PortPin` from the following value. See [Table 8](#) for selectable values.
- If `PortPinOutputInBufEnable` is checked, the input buffer is set to be enabled when the pin direction is `PORT_PIN_OUT`. The actual output level can be read by this setting. If not checked, the input buffer is disabled and the level setting on the hardware can be read.
- `PortPinMode` will be ignored, the parameter `PortPinInitialMode` will be used. Selectable port pin mode is fixed in each subderivative.
- If `PortPinSmartioBypassEnable` is checked, the SMART I/O path is bypassed for this pin's signal. It means the SMART I/O block is not present for this port pin.
- If `PortPinSmartioSyncIoEnable` is checked, the IO pin input signals to the SMART I/O block are synchronized to "clk_fabric", which is configured with `PortSmartioClockSource`.
- If `PortPinSmartioSyncChipEnable` is checked, the chip pin input signals to the SMART I/O block are synchronized to "clk_fabric", which is configured with `PortSmartioClockSource`.
- `PortPinSmartioLutTr0Source` is the source of LUT input signal "tr0_in". See [Table 16](#) for selectable values.
- `PortPinSmartioLutTr1Source` is the source of LUT input signal "tr1_in". See [Table 16](#) for selectable values.
- `PortPinSmartioLutTr2Source` is the source of LUT input signal "tr2_in". See [Table 16](#) for selectable values.
- `PortPinSmartioLut` is the LUT configuration setting. It is used to determine the LUT output signal and the next sequential state.
- `PortPinSmartioLutOperation` is the LUT opcode specifies the LUT operation setting. See [Table 17](#) for selectable values.
- `PortPinInputBufferMode` is the input buffer mode (trip points and hysteresis). This parameter is available for IO cell types other than `HSIO_ENH` and `HSIO_ENH_PDIFF`. See [Table 9](#) for selectable values.
- If `PortPinOutputSlowSlewRateEnable` is checked, this port pin works in slow slew rate. If not checked, this port pin works in fast slew rate.

Note: *`PortPinOutputSlowSlewRateEnable` is editable only if the slow slew rate functionality of the specified pin is valid.*

- `PortPinOutputDriveStrength` is the GPIO drive strength setting. This parameter is available for IO cell types other than `HSIO_STDLN`, `HSIO_ENH` and `HSIO_ENH_PDIFF`. See [Table 10](#) for selectable values.

- `Port5VPinInputBufferMode` is the input buffer mode (trip points and hysteresis) for S40E GPIO upper bit. This parameter is available for IO cell types other than HSIO_ENH and HSIO_ENH_PDIFF. See [Table 11](#) for selectable values.
- `PortPinOutputDriveSelectTrim` is the GPIO drive select trim setting. See [Table 12](#) for selectable values. This parameter is available for Traveo II 2D cluster series only.
- `PortPinOutputSlewExt` is the GPIO output extra slew rate control. See [Table 13](#) for selectable values.

Note: *`PortPinOutputSlewExt` is editable only if the extra slew rate control functionality of the specified pin is valid.*

- `PortPinOutputDriveExt` is the GPIO output extra drive strength control. See [Table 14](#) for selectable values.

Note: *`PortPinOutputDriveExt` is editable only if the extra drive strength functionality of the specified pin is valid.*

4.5 AMUX Splitter Cell Configuration

For the configuration of the AMUX Splitter Cells the following parameters are available:

- `PortAmuxSplitCellId` is the Cell Id of the AMUX Splitter Cell in this `PortAmuxSplitCell`. The range is 0 to (the number of AMUX Splitter Cells - 1).
- `PortAmuxBusaSwitchSL` is the T-switch control for Left AMUXBUSa switch in this `PortAmuxSplitCell`. See [Table 21](#) for selectable values.
- `PortAmuxBusaSwitchSR` is the T-switch control for Right AMUXBUSa switch in this `PortAmuxSplitCell`. See [Table 21](#) for selectable values.
- `PortAmuxBusaSwitchS0` is the T-switch control for AMUXBUSa vssa/ground switch in this `PortAmuxSplitCell`. See [Table 21](#) for selectable values.
- `PortAmuxBusbSwitchSL` is the T-switch control for Left AMUXBUSb switch in this `PortAmuxSplitCell`. See [Table 21](#) for selectable values.
- `PortAmuxBusbSwitchSR` is the T-switch control for Right AMUXBUSb switch in this `PortAmuxSplitCell`. See [Table 21](#) for selectable values.
- `PortAmuxBusbSwitchS0` is the T-switch control for AMUXBUSb vssa/ground switch in this `PortAmuxSplitCell`. See [Table 21](#) for selectable values.

4.6 Port Default Configuration Set

Port Default Configuration is used as default parameters for unconfigured Port Pins and AMUX Splitter Cells. The pins and cells, which are not configured in `PortConfigSet` is initialized with `PortDefaultConfigSet`.

4.7 Port Default Configuration

Port Default Configuration is the default parameters for unconfigured Ports.

- `PortDefSmartioClockSource` is the source of clock ("clk_fabric") and reset ("rst_fabric_n") for the SMART I/O block.
- If `PortDefSmartioHoldOverrideEnable` is checked, the SMART I/O controls the I/O cell hold override functionality for the SMART I/O block. If not checked, it is controlled by the HSIOM.
- If `PortDefSmartioPipelineEnable` is checked, the pipeline setting is enabled for the SMART I/O block.
- If `PortDefSmartioEnable` is checked, the programmable I/O setting is enabled for the SMART I/O block.

- `PortDefSmartioDataUnitTr0Source` is the data unit input signal "tr0_in" source for the SMART I/O block. See [Table 18](#) for selectable values.
- `PortDefSmartioDataUnitTr1Source` is the data unit input signal "tr1_in" source for the SMART I/O block. See [Table 18](#) for selectable values.
- `PortDefSmartioDataUnitTr2Source` is the data unit input signal "tr2_in" source for the SMART I/O block. See [Table 18](#) for selectable values.
- `PortDefSmartioDataUnitData0Source` is the data unit input data "data0_in" source for the SMART I/O block. See [Table 19](#) for selectable values.
- `PortDefSmartioDataUnitData1Source` is the data unit input data "data1_in" source for the SMART I/O block. See [Table 19](#) for selectable values.
- `PortDefSmartioDataUnitBitSize` is the size/width of the data unit data operands (in bits) for the SMART I/O block.
- `PortDefSmartioDataUnitOperation` is the data unit opcode specifies the data unit operation for the SMART I/O block. See [Table 20](#) for selectable values.
- `PortDefSmartioDataUnitSource` is the data unit input data source for the SMART I/O block.

4.8 Port Default Pin Configuration

Port Default Pin Configuration is the default parameters for unconfigured Port Pins.

- `PortDefPinDirection` is the initial direction of unconfigured pins. (IN, OUT or IN/OUT disabled). `PORT_PIN_IN_OUT_DISABLED` is for Analog I/O. It can be set if `PortDefPinInitialMode` is `AMUXA`, `AMUXB`, `AMUXA_DSI` or `AMUXB_DSI`. See [Port_PinDirectionType](#) for selectable values.
- If `PortDefPinDirectionChangeable` is checked, the direction is changeable on unconfigured port pins during runtime.
- `PortDefPinInitialMode` is the initial mode of unconfigured pins.
- If `PortDefPinModeChangeable` is checked, the mode is changeable on unconfigured port pins during runtime.
- `PortDefPinLevelValue` is the level value from unconfigured port pins. See [Port_PinLevelValueType](#) for selectable values.

Note: *When the pin mode is set to other than GPIO, the pin output shall be controlled by the peripheral functionality.*

Therefore, if the initialized pin mode is not GPIO, the configuration parameter `PortDefPinLevelValue` is ignored.

- `PortDefPinOutputDrive` is the output drive mode of unconfigured port pins. See [Table 8](#) for selectable values.
- If `PortDefPinOutputInBufEnable` is checked, the input buffer is set to be enabled when the pin direction is `PORT_PIN_OUT`. The actual output level can be read by this setting. If not checked, the input buffer is disabled and the level setting on the hardware can be read.
- If `PortDefPinSmartioBypassEnable` is checked, the SMART I/O path is bypassed for unconfigured pins' signal. It means the SMART I/O blocks are not present for unconfigured port pins.
- If `PortDefPinSmartioSyncIoEnable` is checked, the I/O pin input signals to the SMART I/O block are synchronized to "clk_fabric", which is configured with `PortDefSmartioClockSource`.
- If `PortDefPinSmartioSyncChipEnable` is checked, the chip pin input signals to the SMART I/O block are synchronized to "clk_fabric", which is configured with `PortDefSmartioClockSource`.

EB tresos Studio Configuration Interface

- `PortDefPinSmartioLutTr0Source` is the source of LUT input signal "tr0_in" on unconfigured port pins. See [Table 16](#) for selectable values.
- `PortDefPinSmartioLutTr1Source` is the source of LUT input signal "tr1_in" on unconfigured port pins. See [Table 16](#) for selectable values.
- `PortDefPinSmartioLutTr2Source` is the source of LUT input signal "tr2_in" on unconfigured port pins. See [Table 16](#) for selectable values.
- `PortDefPinSmartioLut` is the LUT configuration setting. It is used to determine the LUT output signal and the next sequential state.
- `PortDefPinSmartioLutOperation` is the LUT opcode specifies the LUT operation setting on unconfigured port pins. See [Table 17](#) for selectable values.
- `PortDefPinInputBufferMode` is the input buffer mode (trip points and hysteresis) on unconfigured port pins. See [Table 9](#) for selectable values.
- If `PortDefPinOutputSlowSlewRateEnable` is checked, unconfigured port pins work in slow slew rate. If not checked, these port pins work in fast slew rate.
- `PortDefPinOutputDriveStrength` is the GPIO drive strength setting on unconfigured port pins. See [Table 10](#) for selectable values.
- `PortDef5VPinInputBufferMode` is the input buffer mode (trip points and hysteresis) for S40E GPIO upper bit on unconfigured port pins. See [Table 11](#) for selectable values.
- `PortDefPinOutputDriveSelectTrim` is the GPIO drive select trim setting on unconfigured port pins. See [Table 12](#) for selectable values. This parameter is available for Traveo II 2D cluster series only.
- `PortDefPinOutputSlewExt` is the GPIO output extra slew rate control. See [Table 13](#) for selectable values. This parameter is available for Traveo II 2D cluster series only.
- `PortDefPinOutputDriveExt` is the GPIO output extra drive strength control. See [Table 14](#) for selectable values. This parameter is available for Traveo II 2D cluster series only.

4.9 Port Default AMUX Splitter Cell Configuration

Default AMUX Splitter Cell Configuration is the default parameters for unconfigured AMUX Splitter Cells.

- `PortDefAmuxBusaSwitchSL` is the T-switch control for Left AMUXBUSA switch. See [Table 21](#) for selectable values.
- `PortDefAmuxBusaSwitchSR` is the T-switch control for Right AMUXBUSA switch. See [Table 21](#) for selectable values.
- `PortDefAmuxBusaSwitchS0` is the T-switch control for AMUXBUSA *vssa*/ground switch. See [Table 21](#) for selectable values.
- `PortDefAmuxBusbSwitchSL` is the T-switch control for Left AMUXBUSB switch. See [Table 21](#) for selectable values.
- `PortDefAmuxBusbSwitchSR` is the T-switch control for Right AMUXBUSB switch. See [Table 21](#) for selectable values.
- `PortDefAmuxBusbSwitchS0` is the T-switch control for AMUXBUSB *vssa*/ground switch. See [Table 21](#) for selectable values.

4.10 Port Trigger Configuration Set

Port Trigger Configuration is the configuration parameters for Trigger Groups and Trigger 1-to-1 Groups. For more detail about Trigger, see the hardware manual.

- `PortTriggerConfigSetId` is the Id number of *PortTriggerConfigSet*. The range is 0 to (the number of *PortTriggerConfigSet* - 1).

4.11 Port Trigger Group Configuration

Port Trigger Group Configuration is the configuration parameters for Trigger Groups.

- `PortTrGroupId` is the Id of the Trigger Group specified by `PortTrGroupName`.
- `PortTrGroupName` is the name of selected Trigger Group. The selectable Trigger Groups are hardware dependent.
 - The symbolic name is derived from the `PortTrGroupContainer` container short name prefixed with `"PortConf_PortTrGroupContainer_"`.

Note: First, set `PortTrGroupName`. Then press the calculate button of `PortTrGroupId` parameter to get the corresponding `PortTrGroupId`.

4.12 Port Output Trigger Configuration

Port Output Trigger Configuration is the configuration parameters for Output Triggers.

- `PortTrOutputName` is the name of selected Output Trigger. The selectable Output Triggers depend on the Trigger Group in `PortTrGroupName`.
 - The symbolic name is derived from the `PortOutputTrigger` container short name prefixed with `"PortConf_PortOutputTrigger_"`.
- `PortTrInputName` is the name of Input Trigger connected to Output Trigger for this `PortOutputTrigger`. The selectable Input Triggers depend on the Trigger Group in `PortTrGroupName`.
- If `PortTrInvertEnable` is checked, the output trigger invert functionality is enabled in this `PortOutputTrigger`.
- `PortTrSensitiveType` is the output trigger edge sensitive functionality type in this `PortOutputTrigger`. Refer to [Port_TriggerSensitiveType](#) for selectable values.

Note: `PortTrInvertEnable` and `PortTrSensitiveType` are editable only if Trigger Groups have trigger manipulation features.

- If `PortTrDbgFreezeEnable` is checked, the debug freeze functionality is enabled in this `PortOutputTrigger`.

Note: When calling `Port_Init()`, unconfigured Output Triggers are initialized as below:
`PortTrInputName=0`, `PortTrInvertEnable=disable`, `PortTrSensitiveType="Level Sensitive"` and `PortTrDbgFreezeEnable=disable`.

4.13 Port Trigger 1-to-1 Group Configuration

Port Trigger 1-to-1 Group Configuration is the configuration parameters for Trigger 1-to-1 Groups.

- `PortTr1To1GroupId` is the Id of the Trigger 1-to-1 Group specified by `PortTr1To1GroupName`.
- `PortTr1To1GroupName` is the name of selected Trigger 1-to-1 Group. The selectable Trigger 1-to-1 Groups are hardware dependent.
 - The symbolic name is derived from the `PortTr1To1GroupContainer` container short name prefixed with `"PortConf_PortTr1To1GroupContainer_"`.

Note: First, set `PortTr1To1GroupName`. Then press the calculate button of `PortTr1To1GroupId` parameter to get the corresponding `PortTr1To1GroupId`.

4.14 Port 1-to-1 Output Trigger Configuration

Port 1-to-1 Output Trigger Configuration is the configuration parameters for 1-to-1 Output Triggers.

- `PortTr1To1OutputName` is the name of selected 1-to-1 Output Trigger. The selectable 1-to-1 Output Triggers depend on the Trigger 1-to-1 Group in `PortTr1To1GroupName`.
 - The symbolic name is derived from the `Port1To1OutputTrigger` container short name prefixed with “`PortConf_Port1To1OutputTrigger_`”.
- `PortTr1To1InputType` is the input trigger type in this `Port1To1OutputTrigger`. See [Table 22](#) for selectable values.
- If `PortTr1To1InvertEnable` is checked, the output trigger invert functionality is enabled in this `Port1To1OutputTrigger`.
- `PortTr1To1SensitiveType` is the output trigger edge sensitive functionality type in this `Port1To1OutputTrigger`. Refer to [Port_TriggerSensitiveType](#) for selectable values.

Note: *`PortTr1To1InvertEnable` and `PortTr1To1SensitiveType` are editable only if Trigger 1-to-1 Groups have trigger manipulation features.*

- If `PortTr1To1DbgFreezeEnable` is checked, the debug freeze functionality is enabled in this `Port1To1OutputTrigger`.

Note: *When calling `Port_Init()`, unconfigured 1-to-1 Output Triggers are initialized as below:
`PortTr1To1InputType`=Constant signal level '0', `PortTr1To1InvertEnable`=disable,
`PortTr1To1SensitiveType`="Level Sensitive" and `PortTr1To1DbgFreezeEnable`=disable.*

5 Functional Description

5.1 Inclusion

The file *Port.h* includes all necessary external identifiers. Therefore, the application only needs to include *Port.h* to make all API functions and data types available.

5.2 Initialization

The PORT Driver provides an initialization function for initializing the Microcontroller's whole port structure. As it is possible to configure more than one configuration, the `Port_Init` function can be called with different configuration sets. The function `Port_Init` must be called at least once after reset. `Port_Init` can be called several times to reconfigure the ports and port pins of the MCU.

Code Listing 1 Code 1. Example Using the `Port_Init()` Function Using the 1st Configuration Set

```
Port_Init(&Port_Config[0]);
```

Note: The PORT Driver module's environment shall call the function `Port_Init` first in order to initialize the port for use.

If the `Port_Init` function is not called first, then no operation can occur on the MCU ports and port pins.

When the initialized pin direction is output and same as previous direction, `Port_Init` shall keep the pin output.

However, if pin's output configuration are updated with `Port_Init()` call, the output signals may be sent with previous setting in a short period.

To configure SMART I/O, the functionality shall be disabled while `SMARTIO_PRT` registers are updated.

Note that the disabled period shall occur during initialization.

The function `Port_Init` enters/exits critical sections every initialization of each port or trigger group. The duration for initializing a trigger group depends on the number of output triggers. Therefore, if a trigger group has many output triggers, the period of one exclusive area becomes long.

Flash boot configures JTAG reset input pin to `SWJ_TRSTN` mode upon reset. (Refer to device datasheet for exact pin number.) This pin is optional for JTAG debug protocol and can be configured to some other mode. If the debug protocol is JTAG and this pin is being configured to some other mode (E.g. GPIO) via `Port_Init()`, then debug session crashes. This applies only when the debug protocol is JTAG. This issue can be avoided by modifying JTAG reset input to GPIO in the scope of debug script with following register modification sequence before executing `Port_Init()` inside application code.

1) Modify `HSIOM_PRT_SEL` register

2) Modify `GPIO_PRT_CFG` register

5.3 Runtime Reconfiguration

Changing the port pin direction is only available for port pins configured as direction changeable.

Changing the port pin mode is only available for port pins configured as mode changeable.

Note: `Port_Init()` can be used to reconfigure the whole port structure according to the configuration set provided by parameter `ConfigPtr`.

Code Listing 2 Code 2. Example Using the Port_SetPinDirection() Function and Port_SetPinMode() Function

```
#include "Port.h"
/* ... */
/* initialize ports */
/* .... */
/* set pin direction */
Port_SetPinDirection(PortConf_PortPin_MY_IO_PIN, PORT_PIN_IN);
/* set pin mode */
Port_SetPinMode(PortConf_PortPin_MY_IO_PIN, my_pin_mode);
```

The direction of port pins configured as direction unchangeable can be refreshed. All changes at runtime for port directions of port pins that are configured as “direction changeable at runtime” keep their last state.

Code Listing 3 Code 3. Example Using the Port_RefreshPortDirection() Function

```
#include "Port.h"
/* ... */
/* initialize ports */
/* .... */
/* refresh pin direction */
Port_RefreshPortDirection(void);
```

The PORT Driver provides a reconfiguration function for an output trigger setting and a 1-to-1 output trigger setting.

Code Listing 4 Code 4. Example Using the Port_SetTrigger() Function for Output Trigger and 1-to-1 Output Trigger

```
#include "Port.h"
/* ... */
/* initialize ports */
/* .... */
/* set output trigger */
Port_SetTrigger(PortConf_PortTrGroupContainer_PortTrGroupContainer_0,
                PortConf_PortOutputTrigger_PortOutputTrigger_0,
                PORT_TR_GROUP_DW0_TR_IN_CPUSS_DW0_TR_OUT_0, FALSE,
                PORT_TR_SENSITIVE_LEVEL, FALSE);
/* set 1-to-1 output trigger */
Port_SetTrigger(PortConf_PortTr1To1GroupContainer_PortTr1To1GroupContainer_0,
                PortConf_Port1To1OutputTrigger_Port1To1OutputTrigger_0,
                PORT_TR_1TO1_IN_INPUT, FALSE, PORT_TR_SENSITIVE_LEVEL,
                FALSE);
```

5.4 Ports and Port Pins

A port pin represents a single pin of the Microcontroller. The value of a single pin can either be 0 or 1.

A port represents several port pins, which are grouped according to hardware and are controlled by one hardware register. The corresponding data type for a port's value depends on the bit width of the largest port. Writing to a port with a smaller width ignores the upper bits.

Note: The PORT module only handles port pins. An overall grouping or configuring into ports is not possible.

5.5 Trigger Command

Trigger Command provides software control over trigger activation. This is useful for software-initiated triggers or for debugging purposes. The command enables software activation of one specific input trigger or output trigger of the trigger multiplexer setting.

Code Listing 5 Code 5. Example Using the Port_ActTrigger() and Port_DeactTrigger() Function.

```
#include "Port.h"
/* ... */
/* initialize ports */
/* .... */
/* activate trigger command */
Port_ActTrigger(PortConf_PortTrGroupContainer_PortTrGroupContainer_0,
                PortConf_PortOutputTrigger_PortOutputTrigger_0,
                PORT_TR_ACTIVATION_OUTPUT, PORT_TR_SENSITIVE_LEVEL);
/* .... */
/* deactivate trigger command */
Port_DeactTrigger(void);
```

Note: Modules that use the trigger command shall call the Port_GetTriggerCmdStatus function to confirm that the trigger with edge sensitive has been completed.

Note: Modules that use the trigger command shall check the trigger command activation status is deactivated using Port_GetTriggerCmdStatus before calling Port_ActTrigger.

Note: Modules that use the trigger command shall call the Port_DeactTrigger function only if the trigger command is successfully activated with the level sensitive.

5.6 API Parameter Checking

The driver's services perform regular error checks.

When an error occurs, the error hook routine (configured via PortErrorCalloutFunction) is called and the error code, as well as service ID, module ID and instance ID are passed as parameters.

If default error detection is enabled, all errors are also reported to the DET, a central error hook function within the AUTOSAR environment. The checking itself cannot be deactivated for safety reasons.

The following development error checks are performed by the services of the PORT Driver:

Functional Description

- The `Port_Init` function checks whether the parameter configuration pointer `ConfigPtr` is valid.
 - In case of an invalid `ConfigPtr`, the error code `PORT_E_INIT_FAILED` is reported.
- The `Port_SetPinDirection`, `Port_SetPinMode`, `Port_RefreshPortDirection`, `Port_GetStatus`, `Port_SetToDioMode`, `Port_SetToAlternateMode`, `Port_GetAmuxSplitCtlStatus`, `Port_SetTrigger`, `Port_ActTrigger`, `Port_DeactTrigger`, `PortGetTriggerIdStatus` and `Port_GetTriggerCmdStatus` functions check whether the ports and port pins have been initialized. If the module initialization function is not called before, or configuration is invalid, `PORT_E_UNINIT` is reported.
- The `Port_SetPinDirection` function checks whether the parameter pin identifier `Pin` is valid.
 - In case of an invalid `Pin`, the error code `PORT_E_PARAM_PIN` is reported.
 - In case `PortPinDirectionChangeable` is set to unchangeable for this `Pin`, the error code `PORT_E_DIRECTION_UNCHANGEABLE` is reported.
 - When a direction changes to an unknown `Direction` (different from `PORT_PIN_IN`, `PORT_PIN_OUT` or `PORT_PIN_IN_OUT_DISABLED`) is requested, the error code `PORT_E_PARAM_INVALID_DIRECTION` will be reported.
- The `Port_SetPinMode` function checks whether the parameter pin identifier `Pin` is valid.
 - In case of an invalid `Pin`, the error code `PORT_E_PARAM_PIN` is reported.
 - In case `PortPinModeChangeable` is set to unchangeable for this `Pin`, the error code `PORT_E_MODE_UNCHANGEABLE` is reported.
 - In case the new requested `Mode` is not valid, the error code `PORT_E_PARAM_INVALID_MODE` is reported.
- The `Port_GetVersionInfo` function checks whether the parameter version information pointer `versioninfo` is a NULL pointer.
 - In case of a NULL pointer, the error code `PORT_E_PARAM_POINTER` is reported.
- The `Port_GetStatus` function checks whether the parameter port status information pointer `PortStatusInfoPtr` is a NULL pointer.
 - In case of a NULL pointer, the error code `PORT_E_PARAM_POINTER` is reported.
- The `Port_GetStatus` function checks whether the parameter pin identifier `Pin` is valid.
 - In case of an invalid `Pin`, the error code `PORT_E_PARAM_PIN` is reported.
- The `Port_GetStatus` function checks whether the register values are valid.
 - In case of an invalid register value, the error code `PORT_E_REGISTER` is reported.
- The `Port_SetToDioMode` and `Port_SetToAlternateMode` functions check whether the parameter pin identifier `Pin` is valid.
 - In case of an invalid `Pin`, the error code `PORT_E_PARAM_PIN` is reported.
 - In case `PortPinModeChangeable` is set to unchangeable for this `Pin`, the error code `PORT_E_MODE_UNCHANGEABLE` is reported.
- The `Port_GetAmuxSplitCtlStatus` function checks whether the parameter AMUX Splitter Cell status information pointer `AmuxSplitCtlStatusInfoPtr` is a NULL pointer.
 - In case of a NULL pointer, the error code `PORT_E_PARAM_POINTER` is reported.
- The `Port_GetAmuxSplitCtlStatus` function checks whether the parameter cell identifier `Cell` is valid.
 - In case of an invalid `Cell`, the error code `PORT_E_PARAM_CELL` is reported.
- The `Port_SetTrigger`, `Port_ActTrigger`, and `Port_GetTriggerIdStatus` functions check whether the parameter trigger group identifier `group_id` is valid.
 - In case of an invalid `group_id`, the error code `PORT_E_PARAM_TR_GROUP` is reported.
- The `Port_SetTrigger` and `Port_GetTriggerIdStatus` functions check whether the parameter output trigger identifier `out_trg` is valid.

Functional Description

- In case of an invalid `out_trg`, the error code `PORT_E_PARAM_TR_OUTPUT` is reported.
- The `Port_SetTrigger` function checks whether the parameter trigger group identifier `group_id` is being used in Trigger Command.
 - In case Trigger Command is activated with same Trigger Group as this `group_id`, the error code `PORT_E_TR_CMD_STATUS` is reported.
- The `Port_SetTrigger` function checks whether the parameter input trigger identifier `in_trg` is valid.
 - In case of an invalid `in_trg`, the error code `PORT_E_PARAM_TR_INPUT` is reported.
- The `Port_SetTrigger` function checks whether the parameter trigger group identifier `group_id` indicates the trigger group which has trigger manipulation features.
 - In case `group_id` does not have trigger manipulation features when `inv_flg` is true or `sensitive_type` is `PORT_TR_SENSITIVE_EDGE`, the error code `PORT_E_TR_MANIPULATION_NOT_PRESENT` is reported.
- The `Port_SetTrigger` and `Port_ActTrigger` functions check whether the parameter trigger sensitive type `sensitive_type` is valid.
 - In case of an invalid `sensitive_type`, the error code `PORT_E_PARAM_TR_SENSITIVE` is reported.
- The `Port_ActTrigger` function checks whether the parameter trigger activation type `act_type` is valid.
 - In case of an invalid `act_type`, the error code `PORT_E_PARAM_TR_ACTIVATION` is reported.
- The `Port_ActTrigger` function checks whether the parameter trigger identifier `trg_id` is valid for input trigger when the parameter trigger activation type `act_type` is `PORT_TR_ACTIVATION_INPUT`.
 - In case of an invalid `trg_id`, the error code `PORT_E_PARAM_TR_INPUT` is reported.
- The `Port_ActTrigger` function checks whether the parameter trigger identifier `trg_id` is valid for output trigger when the parameter trigger activation type `act_type` is `PORT_TR_ACTIVATION_OUTPUT`.
 - In case of an invalid `trg_id`, the error code `PORT_E_PARAM_TR_OUTPUT` is reported.
- The `Port_ActTrigger` function checks whether the trigger command is deactivated.
 - If the trigger command is activated, the error code `PORT_E_TR_CMD_STATUS` is reported.
- The `Port_DeactTrigger` function checks whether the trigger command is activated with level sensitive.
 - If that trigger command is deactivated or activated with edge sensitive, the error code `PORT_E_TR_CMD_STATUS` is reported.
- The `Port_GetTriggerIdStatus` function checks whether the parameter trigger ID status information pointer `TrigIdStatusInfoPtr` is a NULL pointer.
 - In case of a NULL pointer, the error code `PORT_E_PARAM_POINTER` is reported.
- The `Port_GetTriggerCmdStatus` function checks whether the parameter trigger command status information pointer `TrigCmdStatusInfoPtr` is a NULL pointer.
 - In case of a NULL pointer, the error code `PORT_E_PARAM_POINTER` is reported.

5.7 Configuration Checking

The following conditions are checked when configuring the PORT Driver with EB tresos Studio:

- Multiple occurrence of the same short name for *PortConfigSet* containers

The short name for *PortConfigSet* container must be unique.

- Multiple occurrence of the same short name for *PortContainer* containers

The short name for *PortContainer* container must be unique in the same configuration set.

- The short name for *PortContainer* container is different from *PortContainer* which has same `PortId`

Functional Description

All `PortId` of the container of a same short name should be the same in the different configuration set.

- Multiple occurrence of the same short name for *PortPin* containers

The short name for *PortPin* container must be unique in the same configuration set.

- The short name for *PortPin* container is different from *PortPin* which has same `PortPinId`

All `PortPinId` of the container of a same short name should be the same in the different configuration set.

- Multiple occurrence of same `PortId`

`PortId` must be unique in the same configuration set.

- Multiple occurrence of same `PortPinId`

Each `PortPinId` shall be used once to create a symbolic name and configuration for this pin. It is not possible to configure additional pins (different symbolic name) referring the same `PortPinId`.

- Architecture/derivate specific port pin configuration check

Some basic checks are made for each port pin that the initial mode, direction, level and direction change could be covered by this pin. If the mode is not supported by this pin an error is produced at generation time.

- Multiple occurrence of same `PortPinName`

`PortPinName` has to be unique in the same configuration set.

- Multiple occurrence of the same short name for *PortAmuxSplitCell* containers

The short name for *PortAmuxSplitCell* container must be unique in the same configuration set.

- The short name for *PortAmuxSplitCell* container is different from *PortAmuxSplitCell* which has same `PortAmuxSplitCellId`

All `PortAmuxSplitCellId` of the container of a same short name should be the same in the different configuration set.

- Multiple occurrence of same `PortAmuxSplitCellId`

`PortAmuxSplitCellId` has to be unique in the same configuration set.

- Multiple occurrence of the same short name for *PortTriggerConfigSet* containers

The short name for *PortTriggerConfigSet* container must be unique.

- Multiple occurrence of same `PortTriggerConfigSetId`

`PortTriggerConfigSetId` has to be unique across all Port sequences.

- Multiple occurrence of the same short name for *PortTrGroupContainer* containers

The short name for *PortTrGroupContainer* container must be unique in the same configuration set.

- The short name for *PortTrGroupContainer* container is different from *PortTrGroupContainer* which has same `PortTrGroupId`

All `PortTrGroupId` of the container of a same short name should be the same in the different configuration set.

- Multiple occurrence of same `PortTrGroupName`

Trigger Group Name has to be unique in the same configuration set.

Functional Description

- Multiple occurrence of the same short name for *PortOutputTrigger* containers

The short name for *PortOutputTrigger* container must be unique in the same configuration set.

- The short name for *PortOutputTrigger* container is different from *PortOutputTrigger* which has same `PortTrOutputName`

All `PortTrOutputName` of the container of a same short name should be the same in the different configuration set.

- Multiple occurrence of same `PortTrOutputName`

Output Trigger Name has to be unique in the same configuration set.

- Multiple occurrence of the same short name for *PortTr1To1GroupContainer* containers

The short name for *PortTr1To1GroupContainer* container must be unique in the same configuration set.

- The short name for *PortTr1To1GroupContainer* container is different from *PortTr1To1GroupContainer* which has same `PortTr1To1GroupId`

All `PortTr1To1GroupId` of the container of a same short name should be the same in the different configuration set.

- Multiple occurrence of same `PortTr1To1GroupName`

Trigger 1-to-1 Group Name has to be unique in the same configuration set.

- Multiple occurrence of the same short name for *Port1To1OutputTrigger* containers

The short name for *Port1To1OutputTrigger* container must be unique in the same configuration set.

- The short name for *Port1To1OutputTrigger* container is different from *Port1To1OutputTrigger* which has same `PortTr1To1OutputName`

All `PortTr1To1OutputName` of the container of a same short name should be the same in the different configuration set.

- Multiple occurrence of same `PortTr1To1OutputName`

1-to-1 Output Trigger Name has to be unique in the same configuration set.

5.8 Re-entrancy

The following functions are not re-entrant.

- `Port_Init()`
- `Port_RefreshPortDirection()`
- `Port_ActTrigger()`
- `Port_DeactTrigger()`

All other functions are re-entrant to each other and itself if accessing different pins (independent of a port) or different output triggers (independent of a Trigger Group).

5.9 Debugging Support

The PORT Driver does not support debugging.

6 Hardware Resources

6.1 Ports and Pins

All GPIO pins of the Traveo II Microcontroller can be used as digital input/output using the PORT Driver.

Any port can operate in 8-pin or 1-pin units and all registers can be read or written in 1-bit or 32-bit units. The maximum number of ports and the available mode combination depend on the subderivative. For more details, refer to each Hardware Manual.

6.2 SMART I/O Blocks

Smart I/O blocks allow Boolean operations on signals going to the GPIO pins from the device subsystems or on signals coming into the device. Operation can be synchronous or asynchronous and the blocks operate in low-power modes, such as Deep Sleep.

Several ports have each SMART I/O block. Which ports have this block depends on the subderivative. For more details, refer to each Hardware Manual.

6.3 AMUX Splitter Cells

AMUX splitter cell is capable of grounding, disconnecting or feeding through each AMUXBUS, and can bridge between two AMUXBUS segments in different analog supply domains.

The number of AMUX Splitter Cells depends on the subderivative. For more details, refer to each Hardware Manual.

6.4 Trigger Groups and Trigger 1-to-1 Groups

In general, a trigger input signal indicates the completion of a peripheral action or a peripheral event. A trigger output signal initiates a peripheral action. Trigger Group is a multiplexer-based connectivity group. This type connects a peripheral input trigger to multiple peripheral output triggers. The selection is under software control.

Trigger 1-to-1 Group is a 1-to-1-based connectivity group. This type connects a peripheral input trigger to one specific peripheral output trigger.

The types of Trigger Groups/Trigger 1-to-1 Groups, the types of Output Triggers/1-to-1 Output Triggers for each Trigger Group/Trigger 1-to-1 Group and the types of Input Triggers for each Trigger Group depend on the subderivative. For more details, refer to each Hardware Manual.

6.5 Timer

The PORT Driver does not use any hardware timers.

6.6 Interrupts

The PORT Driver does not use any interrupts.

7 Appendix

7.1 API Reference

7.1.1 Data Types

7.1.1.1 Port_ConfigType

Type

```
typedef struct
```

Description

`Port_ConfigType` defines a structure which holds the PORT Driver configuration set and a pointer to the configuration data for each channel.

7.1.1.2 Port_PinType

Type

```
uint16
```

Description

`Port_PinType` stores an identifier for each pin. It contains the port number of the pin in the upper 13 bits and the pin number within the port in the lower 3 bits.

7.1.1.3 Port_PinModeType

Type

```
uint8
```

Description

`Port_PinModeType` stores the possible different modes that can be changed at runtime. Modes resulting in the same change of HW register are grouped together.

7.1.1.4 Port_PinDirectionType

Type

```
typedef enum
{
    PORT_PIN_IN = 0,
    PORT_PIN_OUT = 1,
    PORT_PIN_IN_OUT_DISABLED = 2
} Port_PinDirectionType
```

Description

This type defines an enum describing the possible directions a port pin can have. Use `PORT_PIN_IN` to set a pin input direction. Use `PORT_PIN_OUT` to set a pin output direction. Use `PORT_PIN_IN_OUT_DISABLED` to disable input and output direction while the pin mode is `AMUXA`, `AMUXB`, `AMUXA_DSI` or `AMUXB_DSI`.

7.1.1.5 Port_PinLevelValueType

Type

```
typedef enum
{
    PORT_PIN_LEVEL_LOW = 0,
    PORT_PIN_LEVEL_HIGH = 1
} Port_PinLevelValueType
```

Description

This type defines an enum describing the possible level value of a port pin. Use PORT_PIN_LEVEL_LOW to set a port pin as low level. Use PORT_PIN_LEVEL_HIGH to set a port pin level as high.

7.1.1.6 Port_StatusType

Type

```
typedef struct
{
    uint8    direction;
    uint8    mode;
    uint8    outputDrive;
    uint8    inputBufferMode;
    uint8    outputSlowSlewRateEnable;
    uint8    outputDriveStrength;
    uint8    inputBufferMode5VPin;
    uint8    outputDriveSelectTrim;
    uint8    outputSlewRateExt;
    uint8    outputDriveStrengthExt;
    uint8    sioPinsOutputBufferMode;
    uint8    sioPinsInputBufferMode;
    uint8    sioPinsInputBufferVrefTripPoint;
    uint8    sioPinsInputBufferVohOutputLevel;
    uint8    sioPinsAnalogDftEnable;
    uint8    smartioBypassEnable;
    uint8    smartioClockSource;
    uint8    smartioHoldOverrideEnable;
    uint8    smartioPipelineEnable;
    uint8    smartioEnable;
    uint8    smartioSyncIoEnable;
    uint8    smartioSyncChipEnable;
    uint8    smartioLutTr0Source;
    uint8    smartioLutTr1Source;
    uint8    smartioLutTr2Source;
    uint8    smartioLut;
    uint8    smartioLutOperation;
    uint8    smartioDataUnitTr0Source;
    uint8    smartioDataUnitTr1Source;
    uint8    smartioDataUnitTr2Source;
    uint8    smartioDataUnitData0Source;
    uint8    smartioDataUnitData1Source;
    uint8    smartioDataUnitBitSize;
    uint8    smartioDataUnitOperation;
    uint8    smartioDataUnitSource;
} Port_StatusType
```

Description

Port_StatusType is a structure for informing the setting of PORT channel read out from the HW register.

Appendix

- direction – Port Pin direction. (see [Port_PinDirectionType](#))
When Port Pin direction is PORT_PIN_OUT and GPIO output drive mode is configured to PORT_PIN_OUT_MODE_HIGHZ, the different pin direction is returned. This is because the hardware status is same as the returned direction. If `PortPinOutputInBufEnable` is true, PORT_PIN_IN is returned. If false, PORT_PIN_IN_OUT_DISABLED is returned.
- mode – Port Pin mode (see [Table 7](#)).
- outputDrive – GPIO output drive mode (see [Table 8](#)).
- inputBufferMode – Input buffer mode (trip points and hysteresis) (see [Table 9](#)).
- outputSlowSlewRateEnable – Setting of the slow slew rate. (Enabled=1U, Disabled=0U)
- outputDriveStrength – GPIO drive strength (see [Table 10](#)).
- inputBufferMode5VPin – Input buffer mode (trip points and hysteresis) for S40E GPIO upper bit (see [Table 11](#)).
- outputDriveSelectTrim – GPIO drive select trim (see [Table 12](#)). If the pin does not have drive select trim function, returns fixed 0x00.
- outputSlewRateExt – Output extra slew rate (see [Table 13](#)). If the pin does not have output extra slew rate control function, returns fixed 0x00.
- outputDriveStrengthExt – Output extra drive strength (see [Table 14](#)). If the pin does not have output extra drive strength control function, returns fixed 0x00.
- sioPinsOutputBufferMode – Output buffer mode. If the pin does not have SIO function, returns fixed 0x00.
- sioPinsInputBufferMode – Input buffer trip-point in single ended input buffer mode. If the pin does not have SIO function, returns fixed 0x00.
- sioPinsInputBufferVrefTripPoint – Reference voltage (Vref) trip-point of the input buffer setting. If the pin does not have SIO function, returns fixed 0x00.
- sioPinsInputBufferVohOutputLevel – Regulated Voh output level and trip point of the input buffer for a specific SIO pin pair. If the pin does not have SIO function, returns fixed 0x00.
- sioPinsAnalogDftEnable – Reserved.
- smartioBypassEnable – Setting of the programmable IO bypass. (Enabled=1U, Disabled=0U) If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioClockSource – Clock ("clk_fabric") and reset ("rst_fabric_n") source (see [Table 15](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioHoldOverrideEnable – Setting of the IO cell hold override functionality. (Enabled=1U, Disabled=0U) If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioPipelineEnable – Setting of the pipeline register. (Enabled=1U, Disabled=0U) If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioEnable – Setting of the programmable IO. (Enabled=1U, Disabled=0U) If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioSyncIoEnable – Setting of the IO pin input signal synchronization to "clk_fabric". (Enabled=1U, Disabled=0U) If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioSyncChipEnable – Setting of the chip input signal synchronization to "clk_fabric". (Enabled=1U, Disabled=0U) If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioLutTr0Source – LUT input signal "tr0_in" source (see [Table 16](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioLutTr1Source – LUT input signal "tr1_in" source. (see [Table 16](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioLutTr2Source – LUT input signal "tr2_in" source (see [Table 16](#)). If the pin does not have SMART I/O function, returns fixed 0x00.

Appendix

- smartioLut – LUT configuration. If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioLutOperation – LUT operation (see [Table 17](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitTr0Source – Data unit input signal "tr0_in" source (see [Table 18](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitTr1Source – Data unit input signal "tr1_in" source (see [Table 18](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitTr2Source – Data unit input signal "tr2_in" source (see [Table 18](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitData0Source – Data unit input data "data0_in" source (see [Table 19](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitData1Source – Data unit input data "data1_in" source (see [Table 19](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitBitSize – Size / width of the data unit data operands (in bits). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitOperation – Data unit operation (see [Table 20](#)). If the pin does not have SMART I/O function, returns fixed 0x00.
- smartioDataUnitSource – Data unit input data source. If the pin does not have SMART I/O function, returns fixed 0x00.

7.1.1.7 Port_AmuxCellType

Type

uint8

Description

Port_AmuxCellType stores the AMUX splitter cell.
Available values are dependent on HW.

7.1.1.8 Port_AmuxSplitCtlStatusType

Type

```
typedef struct
{
    uint8    amuxBusaSwitchSL;
    uint8    amuxBusaSwitchSR;
    uint8    amuxBusaSwitchS0;
    uint8    amuxBusbSwitchSL;
    uint8    amuxBusbSwitchSR;
    uint8    amuxBusbSwitchS0;
} Port_AmuxSplitCtlStatusType
```

Description

Port_AmuxSplitCtlStatusType is a structure for informing the setting of AMUX splitter cell read out from the HW register.

- amuxBusaSwitchSL – T-switch control for Left AMUXBUS switch (see [Table 21](#)).
- amuxBusaSwitchSR – T-switch control for Right AMUXBUS switch (see [Table 21](#)).
- amuxBusaSwitchS0 – T-switch control for AMUXBUS vssa/ground switch (see [Table 21](#)).

Appendix

- amuxBusbSwitchSL – T-switch control for Left AMUXBUSB switch (see [Table 21](#)).
- amuxBusbSwitchSR – T-switch control for Right AMUXBUSB switch (see [Table 21](#)).
- amuxBusbSwitchS0 – T-switch control for AMUXBUSB vssa/ground switch (see [Table 21](#)).

7.1.1.9 Port_TriggerGroupIdType**Type**

uint8

Description

Port_TriggerGroupIdType stores the trigger group Id.

0 - 15: Trigger group

16 - 31: Trigger 1-to-1 group.

Available values are dependent on HW.

7.1.1.10 Port_TriggerIdType**Type**

uint8

Description

Port_TriggerIdType stores the trigger Id type for Output Triggers, Input Triggers and 1-to-1 Output Triggers.

Available values are dependent on HW.

7.1.1.11 Port_TriggerSensitiveType**Type**

```
typedef enum
{
    PORT_TR_SENSITIVE_LEVEL = 0,
    PORT_TR_SENSITIVE_EDGE = 1
} Port_TriggerSensitiveType;
```

Description

This type defines an enum describing the trigger sensitive type.

Use PORT_TR_SENSITIVE_LEVEL to set level sensitive.

Use PORT_TR_SENSITIVE_EDGE to set edge sensitive.

7.1.1.12 Port_TriggerActivationType

Type

```
typedef enum
{
    PORT_TR_ACTIVATION_INPUT = 0,
    PORT_TR_ACTIVATION_OUTPUT = 1
} Port_TriggerActivationType;
```

Description

This type defines an enum describing the trigger activation type.

Use PORT_TR_ACTIVATION_INPUT to set the input trigger.

Use PORT_TR_ACTIVATION_OUTPUT to set the output trigger.

7.1.1.13 Port_TriggerIdStatusType

Type

```
typedef struct
{
    Port_TriggerIdType      trInput;
    boolean                 trInvertEnable;
    Port_TriggerSensitiveType trSensitiveType;
    boolean                 trDbgFreezeEnable;
} Port_TriggerIdStatusType;
```

Description

Port_TriggerIdStatusType is a structure for informing the setting of output trigger / 1-to-1 output trigger read out from the HW register.

- trInput – Input trigger id for output trigger. Input trigger type for 1-to-1 output trigger.
- trInvertEnable – Setting of the trigger invert. (Enabled=1U, Disabled=0U)
- trSensitiveType – Trigger sensitive type.
- trDbgFreezeEnable – Setting of the trigger debug freeze. (Enabled=1U, Disabled=0U)

7.1.1.14 Port_TriggerCmdStatusType

Type

```
typedef struct
{
    Port_TriggerGroupIdType group_id;
    Port_TriggerIdType      trg_id;
    Port_TriggerActivationType act_type;
    Port_TriggerSensitiveType sensitive_type;
    uint8                   activate;
} Port_TriggerCmdStatusType;
```

Description

Port_TriggerCmdStatusType is a structure for informing the setting of trigger command read out from the HW register.

- group_id – Trigger group Id. 0 - 15: Trigger group 16 - 31: Trigger 1-to-1 group.
- trg_id – Output trigger Id or 1-to-1 output trigger Id in case of act_type=PORT_TR_ACTIVATION_OUTPUT. Input trigger Id in case of act_type=PORT_TR_ACTIVATION_INPUT.

Appendix

- act_type – Trigger activation type.
- sensitive_type – Trigger sensitive type.
- activate – Trigger command activation status. (Activated=1U, Deactivated=0U)

7.1.2 Constants

7.1.2.1 Error Codes

The service might return the error codes, show in [Table 2](#), if default error detection is enabled:

Table 2 Error Codes

Name	Value	Description
PORT_E_PARAM_PIN	10	Incorrect Port Pin ID is passed.
PORT_E_DIRECTION_UNCHANGEABLE	11	Port Pin is not configured as changeable.
PORT_E_INIT_FAILED	12	NULL pointer is passed, or parameter is out of range.
PORT_E_PARAM_INVALID_MODE	13	Requested mode is not valid.
PORT_E_MODE_UNCHANGEABLE	14	Mode is not changeable.
PORT_E_UNINIT	15	PORT Driver is not initialized, or configuration is invalid.
PORT_E_PARAM_POINTER	16	NULL pointer is passed.
PORT_E_PARAM_CELL	29	The cell value is invalid.
PORT_E_PARAM_INVALID_DIRECTION	32	Requested direction not valid.
PORT_E_REGISTER	33	The register value and the configuration do not match, or register value for setting the port pin is not valid.
PORT_E_PARAM_TR_GROUP	34	The trigger group id value is invalid.
PORT_E_PARAM_TR_OUTPUT	35	The output trigger id value is invalid.
PORT_E_PARAM_TR_INPUT	36	The input trigger id value is invalid.
PORT_E_PARAM_TR_SENSITIVE	37	The trigger sensitive type value is invalid.
PORT_E_PARAM_TR_ACTIVATION	38	The trigger activation type value is invalid.
PORT_E_TR_CMD_STATUS	39	The trigger command status is invalid.
PORT_E_TR_MANIPULATION_NOT_PRESENT	40	The trigger group does not have trigger manipulation features.

7.1.2.2 Version Information

Table 3 lists the version information published in the Driver's header file.

Table 3 Version Information

Name	Value	Description
PORT_SW_MAJOR_VERSION	Refer to release notes	Major version number
PORT_SW_MINOR_VERSION	Refer to release notes	Minor version number
PORT_SW_PATCH_VERSION	Refer to release notes	Patch version number

7.1.2.3 Module Information

Table 4 Module Information

Name	Value	Description
PORT_MODULE_ID	124	Module ID
PORT_VENDOR_ID	66	Vendor ID

7.1.2.4 API Service IDs

The API Service IDs, listed in **Table 5**, are published in the Driver's header file.

Table 5 API Service IDs

Name	Value	Description
PORT_API_INIT	0x0	Port Initialization Service
PORT_API_SET_PIN_DIRECTION	0x1	Set Pin Direction Service
PORT_API_REFRESH_PORT_DIRECTION	0x2	Refresh Port Direction Service
PORT_API_GET_VERSION_INFO	0x3	Get Version Information Service
PORT_API_SET_PIN_MODE	0x4	Set Pin Mode Service
PORT_API_SET_TRIGGER	0xF7	Set Trigger Service
PORT_API_ACT_TRIGGER	0xF8	Activate Trigger Command Service
PORT_API_DEACT_TRIGGER	0xF9	Deactivate Trigger Command Service
PORT_API_GET_STATUS_TR_ID	0xFA	Get Trigger Status Service
PORT_API_GET_STATUS_TR_CMD	0xFB	Get Trigger Command Status Service
PORT_API_SET_TO_DIO_MODE	0xFC	Set GPIO Mode Service
PORT_API_SET_TO_ALTERNATE_MODE	0xFD	Set Alternate Mode Service
PORT_API_GET_STATUS	0xFE	Get Pin Setting Service
PORT_API_GET_AMUX_CTL_STATUS	0xFF	Get AMUX Splitter Cell Status Service

7.1.2.5 Port Pin Status

The following Port Pin Status are published in the driver's header file:

Table 6 Common

Name	Value	Description
PORT_PIN_STATUS_FAILURE	0xFF	Abnormal value is set.

Table 7 Port Pin Mode

Name	Value	Description
PORT_PIN_MODE_GPIO	0	GPIO controls "out"
PORT_PIN_MODE_GPIO_DSI	1	GPIO controls "out", DSI controls "output enable"
PORT_PIN_MODE_DSI_DSI	2	DSI controls "out" and "output enable"
PORT_PIN_MODE_DSI_GPIO	3	DSI controls "out", GPIO controls "output enable"
PORT_PIN_MODE_AMUXA	4	Analog mux bus A
PORT_PIN_MODE_AMUXB	5	Analog mux bus B
PORT_PIN_MODE_AMUXA_DSI	6	Analog mux bus A, DSI control
PORT_PIN_MODE_AMUXB_DSI	7	Analog mux bus B, DSI control

Other values (8-31) and corresponding modes are hardware dependent. Available pin modes are different for each port pin. For details of each type, see [Hardware Documentation](#).

Table 8 Output Drive

Name	Value	Description
PORT_PIN_OUT_MODE_HIGHZ	0	High impedance
PORT_PIN_OUT_MODE_PULLUP	2	Resistive pull up
PORT_PIN_OUT_MODE_PULLDOWN	3	Resistive pull down
PORT_PIN_OUT_MODE_OD_LOW	4	Open drain, drives low
PORT_PIN_OUT_MODE_OD_HIGH	5	Open drain, drives high
PORT_PIN_OUT_MODE_STRONG	6	Strong
PORT_PIN_OUT_MODE_PULLUP_DOWN	7	Resistive pull up and down

For details of each type, see [Hardware Documentation](#).

Table 9 Input Buffer Mode

Name	Value	Description
PORT_PIN_IN_MODE_CMOS	0	Input buffer compatible with CMOS and I2C interfaces
PORT_PIN_IN_MODE_TTL	1	Input buffer compatible with TTL interfaces

For details of each type, see [Hardware Documentation](#).

Appendix

Table 10 Output Drive Strength

Name	Value	Description
PORT_PIN_OUT_STRENGTH_DEFAULT	0	When the pin is GPIO_STD/GPIO_ENH: GPIO drives current at its max rated spec When the pin is GPIO_STD/GPIO_ENH: GPIO_SMC/HSIO default mode.
PORT_PIN_OUT_STRENGTH_FULL	1	When the pin is GPIO_STD/GPIO_ENH: GPIO drives current at its max rated spec (same meaning as PORT_PIN_OUT_STRENGTH_DEFAULT) When GPIO_SMC/HSIO_STD: GPIO drives current at its max rated spec
PORT_PIN_OUT_STRENGTH_1_2	2	GPIO drives current at 1/2 of its max rated spec
PORT_PIN_OUT_STRENGTH_1_4	3	GPIO drives current at 1/4 of its max rated spec

For details of each type, see [Hardware Documentation](#).

Table 11 Input Buffer Mode for S40E GPIO

Name	Value	Description
PORT_PIN_5V_IN_MODE_CMOS_OR_TTL	0	Input buffer is compatible with CMOS/TTL interfaces. If Input Buffer Mode is PORT_PIN_IN_MODE_CMOS, the digital input buffer mode is set to CMOS. If Input Buffer Mode is PORT_PIN_IN_MODE_TTL, the digital input buffer mode is set to TTL.
PORT_PIN_5V_IN_MODE_AUTO	1	Input buffer is compatible with AUTO (elevated Vil) interfaces.

For details of each type, see [Hardware Documentation](#).

Table 12 Output Drive Select Trim

Name	Value	Description
PORT_PIN_OUT_TRIM_DEFAULT	0	Default (50 ohms)
PORT_PIN_OUT_TRIM_DS_120OHM	1	120 ohms
PORT_PIN_OUT_TRIM_DS_90OHM	2	90 ohms
PORT_PIN_OUT_TRIM_DS_60OHM	3	60 ohms
PORT_PIN_OUT_TRIM_DS_50OHM	4	50 ohms
PORT_PIN_OUT_TRIM_DS_30OHM	5	30 ohms
PORT_PIN_OUT_TRIM_DS_20OHM	6	20 ohms
PORT_PIN_OUT_TRIM_DS_15OHM	7	15 ohms

For details of each type, see [Hardware Documentation](#).

Appendix

Table 13 Output Extra Slew Rate Control

Name	Value	Description
PORT_PIN_OUT_SLEW_EXT_FAST	0	Fastest Slew Rate
PORT_PIN_OUT_SLEW_EXT_SLOW	1	Slowest Slew Rate

For details of each type, see [Hardware Documentation](#).

Table 14 Output Extra Drive Strength Control

Name	Value	Description
PORT_PIN_OUT_STRENGTH_EXT_0	0	Mode = xSPI-266 Performance = up to 133 MHz Voltage Support = 3.00 V-3.60 V Package Support = BGA, TQFP
PORT_PIN_OUT_STRENGTH_EXT_1	1	Mode = xSPI-200 Performance = up to 100 MHz Voltage Support = 3.00 V-3.60 V Package Support = BGA, TQFP
PORT_PIN_OUT_STRENGTH_EXT_2	2	Mode = Graphics Performance = up to 80 MHz Voltage Support = 3.00 V-3.60 V Package Support = BGA
PORT_PIN_OUT_STRENGTH_EXT_3	3	Mode = Ethernet-50 MHz/Graphics-64 MHz Performance = up to 50 MHz for Ethernet, up to 64MHz for Graphics Voltage Support = 3.00 V-3.60 V Package Support = BGA, TQFP for Ethernet. TQFP for Graphics
PORT_PIN_OUT_STRENGTH_EXT_4	4	Mode = SPI-12 MHz/SPI-25 MHz/GPIO Support Performance = up to 25 MHz for SPI, up to 20 MHz for GPIO Voltage Support = 3.00 V-3.60 V for SPI. 4.50 V-5.60 V for GPIO Package Support = BGA, TQFP

For details of each type, see [Hardware Documentation](#).

Table 15 Clock and Reset Source

Name	Value	Description
PORT_SMART_CLK_SRC_IO0	0	Clock / reset sources are io_data_in[0] / '1'.
PORT_SMART_CLK_SRC_IO1	1	Clock / reset sources are io_data_in[1] / '1'.
PORT_SMART_CLK_SRC_IO2	2	Clock / reset sources are io_data_in[2] / '1'.
PORT_SMART_CLK_SRC_IO3	3	Clock / reset sources are io_data_in[3] / '1'.
PORT_SMART_CLK_SRC_IO4	4	Clock / reset sources are io_data_in[4] / '1'.
PORT_SMART_CLK_SRC_IO5	5	Clock / reset sources are io_data_in[5] / '1'.
PORT_SMART_CLK_SRC_IO6	6	Clock / reset sources are io_data_in[6] / '1'.

Appendix

Name	Value	Description
PORT_SMART_CLK_SRC_IO7	7	Clock / reset sources are io_data_in[7] / '1'.
PORT_SMART_CLK_SRC_CHIP0	8	Clock / reset sources are chip_data[0] / '1'.
PORT_SMART_CLK_SRC_CHIP1	9	Clock / reset sources are chip_data[1] / '1'.
PORT_SMART_CLK_SRC_CHIP2	10	Clock / reset sources are chip_data[2] / '1'.
PORT_SMART_CLK_SRC_CHIP3	11	Clock / reset sources are chip_data[3] / '1'.
PORT_SMART_CLK_SRC_CHIP4	12	Clock / reset sources are chip_data[4] / '1'.
PORT_SMART_CLK_SRC_CHIP5	13	Clock / reset sources are chip_data[5] / '1'.
PORT_SMART_CLK_SRC_CHIP6	14	Clock / reset sources are chip_data[6] / '1'.
PORT_SMART_CLK_SRC_CHIP7	15	Clock / reset sources are chip_data[7] / '1'.
PORT_SMART_CLK_SRC_CLK_SM_RST_ACT	16	Clock / reset sources are clk_smartio / rst_sys_act_n
PORT_SMART_CLK_SRC_CLK_SM_RST_DS	17	Clock / reset sources are clk_smartio / rst_sys_dpslp_n
PORT_SMART_CLK_SRC_CLK_SM_RST_DS_HIB	18	Same as <i>PORT_SMART_CLK_SRC_CLK_SM_RST_DS</i> . (See Hardware Documentation)
PORT_SMART_CLK_SRC_CLK_LF_RST_LF_DS	19	Clock / reset sources are clk_lf/rst_lf_dpslp_n (note that "clk_lf" is available in DeepSleep power mode)
PORT_SMART_CLK_SRC_CONST0	20	Clock / reset sources are constant '0'
PORT_SMART_CLK_SRC_ASYNC	31	Clock / reset sources are asynchronous mode / '1'

For details of each type, see [Hardware Documentation](#).

Table 16 LUT Input Signal Source

Name	Value	Description
PORT_SMART_LUT_TR_DU	0	Data unit output (for "tr0_in")
PORT_SMART_LUT_TR_LUT0	0	LUT 0 output (for "tr1_in" or "tr2_in")
PORT_SMART_LUT_TR_LUT1	1	LUT 1 output
PORT_SMART_LUT_TR_LUT2	2	LUT 2 output
PORT_SMART_LUT_TR_LUT3	3	LUT 3 output
PORT_SMART_LUT_TR_LUT4	4	LUT 4 output
PORT_SMART_LUT_TR_LUT5	5	LUT 5 output
PORT_SMART_LUT_TR_LUT6	6	LUT 6 output
PORT_SMART_LUT_TR_LUT7	7	LUT 7 output
PORT_SMART_LUT_TR_CHIP04	8	chip_data[0] (for LUTs 0, 1, 2, 3) chip_data[4] (for LUTs 4, 5, 6, 7)
PORT_SMART_LUT_TR_CHIP15	9	chip_data[1] (for LUTs 0, 1, 2, 3) chip_data[5] (for LUTs 4, 5, 6, 7)

Appendix

Name	Value	Description
PORT_SMART_LUT_TR_CHIP26	10	chip_data[2] (for LUTs 0, 1, 2, 3) chip_data[6] (for LUTs 4, 5, 6, 7)
PORT_SMART_LUT_TR_CHIP37	11	chip_data[3] (for LUTs 0, 1, 2, 3) chip_data[7] (for LUTs 4, 5, 6, 7)
PORT_SMART_LUT_TR_IO04	12	io_data_in[0] (for LUTs 0, 1, 2, 3) io_data_in[4] (for LUTs 4, 5, 6, 7)
PORT_SMART_LUT_TR_IO15	13	io_data_in[1] (for LUTs 0, 1, 2, 3) io_data_in[5] (for LUTs 4, 5, 6, 7)
PORT_SMART_LUT_TR_IO26	14	io_data_in[2] (for LUTs 0, 1, 2, 3) io_data_in[6] (for LUTs 4, 5, 6, 7)
PORT_SMART_LUT_TR_IO37	15	io_data_in[3] (for LUTs 0, 1, 2, 3) io_data_in[7] (for LUTs 4, 5, 6, 7)

For details of each type, see [Hardware Documentation](#).

Table 17 LUT Opcode

Name	Value	Description
PORT_SMART_LUT_OPC_COM_OUT_NO_FB	0	Combinatorial output, no feedback
PORT_SMART_LUT_OPC_COM_OUT_FB	1	Combinatorial output, feedback
PORT_SMART_LUT_OPC_SEQ_OUT_NO_FB	2	Sequential output, no feedback
PORT_SMART_LUT_OPC_REG_ASYNC	3	Register with asynchronous set and reset

For details of each type, see [Hardware Documentation](#).

Table 18 Data Unit Input Signal Source

Name	Value	Description
PORT_SMART_DU_TR_CONST0	0	Constant '0'
PORT_SMART_DU_TR_CONST1	1	Constant '1'
PORT_SMART_DU_TR_DU	2	Data unit output
PORT_SMART_DU_TR_LUT0	3	LUT 0 output
PORT_SMART_DU_TR_LUT1	4	LUT 1 output
PORT_SMART_DU_TR_LUT2	5	LUT 2 output
PORT_SMART_DU_TR_LUT3	6	LUT 3 output
PORT_SMART_DU_TR_LUT4	7	LUT 4 output
PORT_SMART_DU_TR_LUT5	8	LUT 5 output
PORT_SMART_DU_TR_LUT6	9	LUT 6 output
PORT_SMART_DU_TR_LUT7	10	LUT 7 output

For details of each type, see [Hardware Documentation](#).

Appendix

Table 19 Data Unit Input Data Source

Name	Value	Description
PORT_SMART_DU_DATA_CONST0	0	Constant "0"
PORT_SMART_DU_DATA_CHIP	1	chip_data[7:0]
PORT_SMART_DU_DATA_IO	2	io_data_in[7:0]
PORT_SMART_DU_DATA_DATA_REG	3	DATA.DATA MMIO register field

For details of each type, see Section [Hardware Documentation](#).

Table 20 Data Unit Opcode

Name	Value	Description
PORT_SMART_DU_OPC_INCR	1	INCR
PORT_SMART_DU_OPC_DECR	2	DECR
PORT_SMART_DU_OPC_INCR_WRAP	3	INCR_WRAP
PORT_SMART_DU_OPC_DECR_WRAP	4	DECR_WRAP
PORT_SMART_DU_OPC_INCR_DECR	5	INCR_DECR
PORT_SMART_DU_OPC_INCR_DECR_WRAP	6	INCR_DECR_WRAP
PORT_SMART_DU_OPC_ROR	7	ROR
PORT_SMART_DU_OPC_SHR	8	SHR
PORT_SMART_DU_OPC_AND_OR	9	AND_OR
PORT_SMART_DU_OPC_SHR_MAJ3	10	SHR_MAJ3
PORT_SMART_DU_OPC_SHR_EQL	11	SHR_EQL

For details of each type, see [Hardware Documentation](#).

7.1.2.6 AMUX Splitter Cell Status

The following AMUX Splitter Cell Status is published in the driver's header file:

Table 21 AMUX T-switch

Name	Value	Description
PORT_AMUX_SWITCH_OPEN	0	AMUX T-switch is open
PORT_AMUX_SWITCH_CLOSED	1	AMUX T-switch is closed

For details of each type, see [Hardware Documentation](#).

7.1.2.7 Trigger Status

The following Trigger Status is published in the driver's header file:

Table 22 Trigger 1-to-1 Input Type

Name	Value	Description
PORT_TR_1TO1_IN_CONST0	0	Constant signal level '0'.
PORT_TR_1TO1_IN_INPUT	1	Input trigger.

For details of each type, see [Hardware Documentation](#).

Appendix

7.1.3 Functions

7.1.3.1 Port_Init

Syntax

```
void Port_Init  
(  
    const Port_ConfigType* ConfigPtr  
)
```

Service ID

0x0

Parameters (in)

- `ConfigPtr` – Pointer to configuration set.

Parameters (out)

None

Return value

None

DET Errors

- `PORT_E_INIT_FAILED` – NULL Pointer is given to this function, or `ConfigPtr` is not the pointer to the configuration information of Port module.

DEM Errors

None

Description

This service initializes all ports and port pins, AMUX splitter cells, trigger groups and trigger command with the configuration set.

7.1.3.2 Port_SetPinDirection

Syntax

```
void Port_SetPinDirection  
(  
    Port_PinType          Pin,  
    Port_PinDirectionType Direction  
)
```

Service ID

0x1

Parameters (in)

- `Pin` – Port Pin ID number.
- `Direction` – Port Pin direction.

Appendix

Parameters (out)

None

Return value

None

DET Errors

- *PORT_E_PARAM_PIN* – The Port Pin ID is invalid.
- *PORT_E_DIRECTION_UNCHANGEABLE* – Port Pin is configured as direction unchangeable.
- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_INVALID_DIRECTION* – The Direction requested is not valid.

DEM Errors

None

Description

The service sets the port pin direction during runtime.

7.1.3.3 Port_RefreshPortDirection

Syntax

```
void Port_RefreshPortDirection  
(  
    void  
)
```

Service ID

0x2

Parameters (in)

None

Parameters (out)

None

Return value

None

DET Errors

- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.

DEM Errors

None

Description

The service refreshes the directions of pins whose directions are configured as not changeable.

Appendix

7.1.3.4 Port_GetVersionInfo

Syntax

```
void Port_GetVersionInfo  
(  
    Std_VersionInfoType* versioninfo  
)
```

Service ID

0x3

Parameters (in)

None

Parameters (out)

- `versioninfo` – Pointer to where to store the version information of this module.

Return value

None

DET Errors

- `PORT_E_PARAM_POINTER` – Parameter `versioninfo` is a NULL pointer.

DEM Errors

None

Description

This service returns module version, vendor and module ID information of the PORT module.

7.1.3.5 Port_SetPinMode

Syntax

```
void Port_SetPinMode  
(  
    Port_PinType      Pin,  
    Port_PinModeType Mode  
)
```

Service ID

0x4

Parameters (in)

- `Pin` – Port Pin ID number.
- `Mode` – Port Pin mode.

Parameters (out)

None

Return value

None

Appendix**DET Errors**

- *PORT_E_PARAM_PIN* – The Port Pin ID is invalid.
- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_INVALID_MODE* – The Mode is out of range.
- *PORT_E_MODE_UNCHANGEABLE* – Port Pin is configured as mode unchangeable.

DEM Errors

None

Description

This service changes the current mode of the port pin to the new one.

7.1.3.6 Port_GetStatus**Syntax**

```
void Port_GetStatus
(
    Port_PinType      Pin,
    Port_StatusType*  PortStatusInfoPtr
)
```

Service ID

0xFE

Parameters (in)

- *Pin* – Port Pin ID number.

Parameters (out)

- *PortStatusInfoPtr* – Where to place the port pin status information.

Return value

None

DET Errors

- *PORT_E_PARAM_POINTER* – Parameter *PortStatusInfoPtr* is a NULL pointer.
- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_PIN* – The Port Pin ID is invalid.
- *PORT_E_REGISTER* – The register value is invalid.

DEM Errors

None

Description

This service returns the hardware register status of the specified pin.

7.1.3.7 Port_GetAmuxSplitCtlStatus

Syntax

```
void Port_GetAmuxSplitCtlStatus
(
    Port_AmuxCellType      Cell,
    Port_AmuxSplitCtlStatusType* AmuxSplitCtlStatusInfoPtr
)
```

Service ID

0xFF

Parameters (in)

- `Cell` – The id of AMUX splitter cell.

Parameters (out)

- `AmuxSplitCtlStatusInfoPtr` – Where to place the AMUX splitter cell status information.

Return value

None

DET Errors

- `PORT_E_UNINIT` – The ports are not initialized, or configuration is invalid.
- `PORT_E_PARAM_POINTER` – Parameter `AmuxSplitCtlStatusInfoPtr` is a NULL pointer.
- `PORT_E_PARAM_CELL` – The id of AMUX splitter cell is invalid.

DEM Errors

None

Description

This service returns the hardware register status of the specified AMUX splitter cell.

7.1.3.8 Port_SetToDioMode

Syntax

```
void Port_SetToDioMode
(
    Port_PinType      PortId
)
```

Service ID

0xFC

Parameters (in)

- `PortId` – Port Pin ID number.

Parameters (out)

None

Appendix**Return value**

None

DET Errors

- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_PIN* – The Port Pin ID is invalid.
- *PORT_E_MODE_UNCHANGEABLE* – Port Pin is configured as mode unchangeable.

DEM Errors

None

Description

This service switches a Port Pin from the configured mode to DIO mode (PORT_PIN_MODE_GPIO).

Note: If the configured mode of a pin is “Dio”, this API has no effect.

7.1.3.9 Port_SetToAlternateMode

Syntax

```
void Port_SetToAlternateMode
(
    Port_PinType      PortId
)
```

Service ID

0xFD

Parameters (in)

- *PortId* – Port Pin ID number.

Parameters (out)

None

Return value

None

DET Errors

- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_PIN* – The Port Pin ID is invalid.
- *PORT_E_MODE_UNCHANGEABLE* – Port Pin is configured as mode unchangeable.

DEM Errors

None

Description

This service switches a Port Pin from DIO mode (PORT_PIN_MODE_GPIO) to the configured mode.

Note: If the configured mode of a pin is “Dio”, this API has no effect.

7.1.3.10 Port_SetTrigger

Syntax

```
void Port_SetTrigger
(
    Port_TriggerGroupIdType    group_id,
    Port_TriggerIdType         out_trg,
    Port_TriggerIdType         in_trg,
    boolean                    inv_flg,
    Port_TriggerSensitiveType   sensitive_type,
    boolean                    dbg_freeze_flg
)
```

Service ID

0xF7

Parameters (in)

- `group_id` – Trigger Group ID number.
- `out_trg` – Output Trigger ID number.
- `in_trg` – Input Trigger ID number in case that `group_id` is Trigger Group. Input Trigger Type in case that `group_id` is Trigger 1-to-1 Group.
- `inv_flg` – Trigger Invert Flag.
- `sensitive_type` – Trigger Sensitive Type.
- `dbg_freeze_flg` – Debug Freeze Flag.

Parameters (out)

None

Return value

None

DET Errors

- `PORT_E_UNINIT` – The ports are not initialized, or configuration is invalid.
- `PORT_E_PARAM_TR_GROUP` – The Trigger Group ID is invalid.
- `PORT_E_PARAM_TR_OUTPUT` – The Output Trigger ID is invalid.
- `PORT_E_PARAM_TR_INPUT` – The Input Trigger ID or Input Trigger Type is invalid.
- `PORT_E_TR_CMD_STATUS` – Trigger Command is being activated with the Trigger Group ID.
- `PORT_E_PARAM_TR_SENSITIVE` – The Trigger Sensitive Type is invalid.
- `PORT_E_TR_MANIPULATION_NOT_PRESENT` – The Trigger Group does not have trigger manipulation features.

DEM Errors

None

Description

This service sets the configuration for specified trigger.

7.1.3.11 Port_ActTrigger

Syntax

```
boolean Port_ActTrigger
(
    Port_TriggerGroupIdType    group_id,
    Port_TriggerIdType         trg_id,
    Port_TriggerActivationType act_type,
    Port_TriggerSensitiveType  sensitive_type
)
```

Service ID

0xF8

Parameters (in)

- `group_id` – Trigger Group ID number.
- `trg_id` – Trigger ID number.
- `act_type` – Trigger Activation Type.
- `sensitive_type` – Trigger Sensitive Type.

Parameters (out)

None

Return value

- *TRUE* - Activation success.
- *FALSE* - Activation failure.

DET Errors

- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_TR_GROUP* – The Trigger Group ID is invalid.
- *PORT_E_PARAM_TR_ACTIVATION* – The Trigger Activation Type is invalid.
- *PORT_E_PARAM_TR_SENSITIVE* – The Trigger Sensitive Type is invalid.
- *PORT_E_PARAM_TR_OUTPUT* – The Trigger ID is invalid for output trigger when parameter `act_type` is *PORT_TR_ACTIVATION_OUTPUT*.
- *PORT_E_PARAM_TR_INPUT* – The Trigger ID is invalid for input trigger when parameter `act_type` is *PORT_TR_ACTIVATION_INPUT*.
- *PORT_E_TR_CMD_STATUS* – Trigger Command is being activated.

DEM Errors

None

Description

This service activates the specified trigger.

Appendix

7.1.3.12 Port_DeactTrigger

Syntax

```
void Port_DeactTrigger
(
    void
)
```

Service ID

0xF9

Parameters (in)

None

Parameters (out)

None

Return value

None

DET Errors

- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_TR_CMD_STATUS* – The trigger command is being deactivated or activated with edge sensitive.

DEM Errors

None

Description

This service deactivates the trigger activated by `Port_ActTrigger`.

This function must be called only if the trigger command is successfully activated with the level sensitive.

7.1.3.13 Port_GetTriggerIdStatus

Syntax

```
void Port_GetTriggerIdStatus
(
    Port_TriggerGroupIdType  group_id,
    Port_TriggerIdType       out_trg,
    Port_TriggerIdStatusType* TrigIdStatusInfoPtr
)
```

Service ID

0xFA

Parameters (in)

- `group_id` – Trigger Group ID number.
- `out_trg` – Output Trigger ID number.

Parameters (out)

- `TrigIdStatusInfoPtr` – Where to place the output trigger status information.

Appendix**Return value**

None

DET Errors

- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_POINTER* – Parameter `TrigIdStatusInfoPtr` is a NULL pointer.
- *PORT_E_PARAM_TR_GROUP* – The Trigger Group ID is invalid.
- *PORT_E_PARAM_TR_OUTPUT* – The Output Trigger ID is invalid.

DEM Errors

None

Description

This service returns the hardware register status of the specified output trigger or 1-to-1 output trigger.

7.1.3.14 Port_GetTriggerCmdStatus**Syntax**

```
void Port_GetTriggerCmdStatus  
(  
    Port_TriggerCmdStatusType* TrigCmdStatusInfoPtr  
)
```

Service ID

0xFB

Parameters (in)

None

Parameters (out)

- `TrigCmdStatusInfoPtr` – Where to place the trigger command status information.

Return value

None

DET Errors

- *PORT_E_UNINIT* – The ports are not initialized, or configuration is invalid.
- *PORT_E_PARAM_POINTER* – Parameter `TrigCmdStatusInfoPtr` is a NULL pointer.

DEM Errors

None

Description

This service gets the status of trigger command.

7.1.4 Required Callback Functions

7.1.4.1 DET

If default error detection is enabled, the PORT Driver uses the following callback function that is provided by DET. If you do not use DET, you must implement this function within your application.

7.1.4.1.1 Det_ReportError

Syntax

```
Std_ReturnType Det_ReportError
(
    uint16 ModuleId,
    uint8 InstanceId,
    uint8 ApiId,
    uint8 ErrorId
)
```

Reentrancy

Reentrant

Parameters (in)

- `ModuleId` – Module ID of calling module.
- `InstanceId` – Instance ID of calling module.
- `ApiId` – ID of the API service that calls this function.
- `ErrorId` – ID of the detected development error.

Return value

Returns always `E_OK` (is required for services).

Description

Service for reporting development errors.

7.1.4.2 Callout functions

7.1.4.2.1 Error callout API

The AUTOSAR PORT module requires an error callout handler. Each error is reported to this handler, error checking cannot be switched off. The name of the function to be called can be configured by parameter `PortErrorCalloutFunction`.

Syntax

```
void Error_Handler_Name
(
    uint16 ModuleId,
    uint8 InstanceId,
    uint8 ApiId,
    uint8 ErrorId
)
```

Reentrancy

Reentrant

Appendix

Parameters (in)

- `ModuleId` – Module ID of calling module.
- `InstanceId` – Instance ID of calling module.
- `ApiId` – ID of the API service that calls this function.
- `ErrorId` – ID of the detected error.

Return value

None

Description

Service for reporting errors.

8 Appendix

8.1 Access Register Table

8.1.1 HSIOM

Table 23 Table B-1. HSIOM Access Register Table

Register	Bit No.	Access Size	Value	Description	Timing	Mask Value	Monitoring Value
HSIOM_PRT_PORT_SEL0 (Port selection 0)	31:0	Word (32 bits)	Depends on configuration value or API.	The hardware peripheral connection (pin mode) to IO pins 0-3.	Port_Init Port_SetPinMode Port_SetToDioMode Port_SetToAlternateMode Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
HSIOM_PRT_PORT_SEL1 (Port selection 1)	31:0	Word (32 bits)	Depends on configuration value or API.	The hardware peripheral connection (pin mode) to IO pins 4-7.	Port_Init Port_SetPinMode Port_SetToDioMode Port_SetToAlternateMode Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
HSIOM_AMUX_SPLIT_CTL (AMUX splitter cell control)	31:0	Word (32 bits)	Depends on configuration value or API.	This register controls the breaking of AMUX buses A and B into multiple segments.	Port_Init Port_GetAmuxSplitCtlStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)

8.1.2 GPIO

Table 24 **Table B-2. GPIO Access Register Table**

Register	Bit No.	Access Size	Value	Description	Timing	Mask Value	Monitoring Value
GPIO_PRT_OUT (Port output data register)	31:0	Word (32 bits)	Depends on configuration value or API	The output data for the IO pins in the port.	Port_Init	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_CFG (Port configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	Configuration of drive mode and input buffer enable for each pin.	Port_Init Port_RefreshPortDirection Port_SetPinDirection Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_CFG_IN (Port input buffer configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	The input buffer for each pin. This register is common for S40S & S40E GPIO pins.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_CFG_OUT (Port output buffer configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	The output driver for each pin.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_CFG_SIO (Port SIO configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	The features that are specific to SIO ports.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_CFG_IN_AUTOLVL (Port S40E GPIO input buffer configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	The S40E GPIO input buffer upper bit for each pin.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_CFG_OUT2 (Port output buffer configuration register 2)	31:0	Word (32 bits)	Depends on configuration value or API	The output driver for each pin. This register is for Traveo	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)

Register	Bit No.	Access Size	Value	Description	Timing	Mask Value	Monitoring Value
				II 2D cluster series only.			
CFG_SLEW_EXT (Port output buffer slew extension configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	Controls the slew rate for each pin. This register is for Traveo II 2D cluster series only.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
CFG_DRIVE_EXT0 (Port output buffer drive extension configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	The output driver for each pin. This register is for Traveo II 2D cluster series only.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
CFG_DRIVE_EXT1 (Port output buffer drive extension configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	The output driver for each pin. This register is for Traveo II 2D cluster series only.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)

8.1.3 SMARTIO

Table 25 Table B-3. SMARTIO Access Register Table

Register	Bit No.	Access Size	Value	Description	Timing	Mask Value	Monitoring Value
SMARTIO_PRT_CTL (Control register)	31:0	Word (32 bits)	Depends on configuration value or API	This register controls programmable IO port.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
SMARTIO_PRT_SYNC_CTL (Synchronization control register)	31:0	Word (32 bits)	Depends on configuration value or API	This register controls synchronization setting for each IO pin.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
SMARTIO_PRT_LUT_SELx (LUT component input selection for LUT x)	31:0	Word (32 bits)	Depends on configuration value or API	LUT input signal source for LUT x (x is 0 to 7)	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
SMARTIO_PRT_LUT_CTLx (LUT component control register for LUT x)	31:0	Word (32 bits)	Depends on configuration value or API	Determines the LUT output signal for LUT x (x is 0 to 7)	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
SMARTIO_PRT_DU_SEL (Data unit component input selection)	31:0	Word (32 bits)	Depends on configuration value or API	Data unit input signal/data source.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
SMARTIO_PRT_DU_CTL (Data unit component control register)	31:0	Word (32 bits)	Depends on configuration value or API	This register selects size/width of the data unit data operands.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
SMARTIO_PRT_DATA (Data register)	31:0	Word (32 bits)	Depends on configuration value or API	Data unit input data source.	Port_Init Port_GetStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)

8.1.4 PERI

Table 26 **Table B-4. PERI Access Register Table**

Register	Bit No.	Access Size	Value	Description	Timing	Mask Value	Monitoring Value
PERI_TR_CMD (Trigger command)	31:0	Word (32 bits)	Depends on configuration value or API	This register provides SW control over trigger activation.	Port_Init Port_ActTrigger Port_DeactTrigger Port_SetTrigger Port_GetTriggerCmdStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
PERI_TR_GR_TR_CTL (Trigger control register)	31:0	Word (32 bits)	Depends on configuration value or API	This register specifies the input trigger for a specific output trigger.	Port_Init Port_SetTrigger Port_GetTriggerIdStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
PERI_TR_1TO1_GR_TR_CTL (Trigger control register)	31:0	Word (32 bits)	Depends on configuration value or API	This register specifies 1-to-1 triggers.	Port_Init Port_SetTrigger Port_GetTriggerIdStatus	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)

Revision history

Revision	Issue Date	Description of Change
**	05/17/2018	New Spec.
*A	09/27/2018	Added two Traveo II Automotive Body Controller High Family TRMs in Hardware Documentation. Deleted the data sheet in Hardware Documentation. Updated register name in Access Register Table according to Hardware Documentation.
*B	06/20/2019	Removed unnecessary registers from 8.1.2 GPIO Supported cluster device Updated hardware documentation information. Updated link number for Layered Software Architecture. Added the limitation for Port_Init function when JTAG reset input pin mode is changed Updated the description in Table A-8. Updated macro name and description in Table A-10.
*C	11/29/2019	Removed unnecessary definition from Table A-9 Output Drive Strength. Updated definition values in Table A-9 Output Drive Strength.
*D	09/07/2020	Changed a memmap file include folder in section "Memory Mapping". Updated definition values in Table A 9. Output Drive Strength. Added description for new configurations in 4.4 Port Pin Configuration, 4.8 Port Default Pin Configuration. Added configuration table in Table A-12, Table A-13. Added description for pin doesn't have specific function in A.1.1 Data Types Port_StatusType. Added registers in B.1.2 GPIO.
*E	2020-11-20	Updated to Infineon template.
*F	2021-02-26	Added description for pins don't have specific function in 4.4 Port Pin Configuration. Removed "reserve" from 7.1.1.8 Port_AmuxSplitCtlStatusType Port_AmuxSplitCtlStatusType.
*G	2021-03-24	Removed "reserve" from 7.1.1.14 Port_TriggerCmdStatusType.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition 2021-03-24

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2021 Infineon Technologies AG.

All Rights Reserved.

Do you have a question about this document?

Go to www.cypress.com/support

Document reference

002-23344 Rev. *G

IMPORTANT NOTICE

The information given in this document shall in no event be regarded as a guarantee of conditions or characteristics ("Beschaffheitsgarantie").

With respect to any examples, hints or any typical values stated herein and/or any information regarding the application of the product, Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind, including without limitation warranties of non-infringement of intellectual property rights of any third party.

In addition, any information given in this document is subject to customer's compliance with its obligations stated in this document and any applicable legal requirements, norms and standards concerning customer's products and any use of the product of Infineon Technologies in customer's applications.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer's technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (www.infineon.com).

WARNINGS

Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies' products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.